

OPSCALE: Operator-level Provisioning and Autoscaling for LLM Serving

Xingqi Cui
Rice University

Chieh-Jan Mike Liang
Microsoft Research

Ziang Tang
Rice University

Jiarong Xing
Rice University

Haoran Qiu
Microsoft Azure Research

Abstract

Achieving cost efficiency while meeting strict user-facing SLOs (e.g., time-to-first-token) remains a fundamental challenge for cloud GPU clusters serving large language models (LLMs). Autoscaling is the key mechanism for cluster resource management, yet a basic system design question is open for serving LLMs: *what should be the unit of scaling?* Existing approaches primarily treat the entire model as a monolithic scaling unit—simple but unable to capture the fine-grained dynamics of inference workloads. As a result, such coarse-grained scaling often leads to either SLO violations under bursty demand or significant GPU under-utilization.

Our characterization reveals substantial operator heterogeneity, exposing operator-level elasticity as a viable scaling primitive. We present OPSCALE, a practical **operator-level orchestration** framework of profiling, provisioning, placement, and runtime serving. OPSCALE is designed to tackle the high complexity and the space explosion problem, arising from operating at this finer granularity. Evaluated with production traces on up to 40 A100s and 24 GB200s, OPSCALE attains SLOs with up to 36.3% fewer GPUs and 28% less power, or achieves 44% higher throughput under fixed cost budgets.

1 Introduction

Online serving of large generative models such as LLMs is the most resource-intensive workload in cloud infrastructure today. Providers must navigate a fundamental tension: meeting strict user-facing Service-level Objectives (SLOs) for Time-to-First-Token (TTFT) and Time-Between-Tokens (TBT), while minimizing GPU footprints and power consumption [57].

Motivation. To avoid SLO violations, providers often statically provision for peak demand. However, based on experience from global cloud providers, LLM inference workloads are highly dynamic with hard-to-predict input/output sequence lengths; thus, reserved GPU pools can only be utilized at 50% and 39% when provisioning for the P95 demand of text and multimodal workloads, respectively [53, 57, 65].

Achieving high utilization requires SLO-aware *autoscaling* [30, 53, 57]. Today’s state-of-the-art operates at *model-level granularity* (Figure 1, top), scaling the number of full model replicas (e.g., prefill instances). This coarse-grained approach is fundamentally inefficient for two reasons.

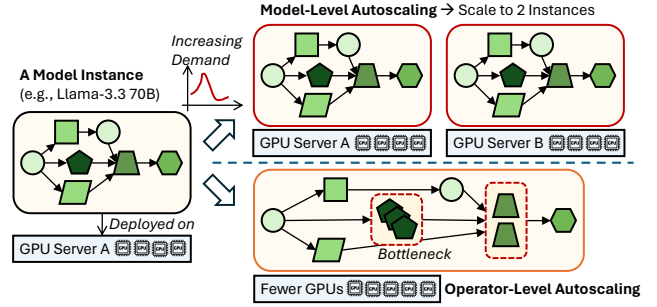


Figure 1: LLM serving at model- vs. operator-level.

First, model-level scaling is resource-inefficient. Generative models are data flows of heterogeneous operators (e.g., attention, linear transformation, and normalization). Forcing all operators to scale uniformly causes *over-provisioning*, where non-bottleneck operators consume precious GPU memory and cycles. Second, model-level autoscaling is slow. Loading a full model onto additional GPUs incurs significant startup latency (e.g., average startup time for a 70B model is around ten seconds even with state-of-the-art methods [16]). This delay makes it largely exceed the time granularity of traffic fluctuations, which often occur within seconds in LLM workloads [53, 57]. Therefore, it either results in SLO violations or costly over-allocation and thus poor cluster utilization.

Our Work. We propose a new paradigm: **operator-level** provisioning and scaling (Figure 1, bottom), which is the key to unblock the two fundamental efficiency limitations. By shifting the unit of scaling from monolithic models to individual operators within each model replica, we enable precise resource allocation and reduce elasticity latency down to sub-second scales. However, realizing this paradigm to achieve resource management at such fine granularity requires addressing two systems challenges.

First, identifying which operators to scale is non-trivial: (1) The inference bottleneck shifts dynamically with workload changes (e.g., arrival rate, input lengths). For example, linear operators dominate compute at short sequence lengths, whereas attention operators become the primary bottleneck under long-context workloads; (2) Not all operators benefit

equally from the same resource scaling action. For instance, linear operators gain from added replicas as the arrival rate grows while operators like normalization are lightweight in compute, and benefit more from batching. Second, scaling operators to separate GPUs wastes resources and incurs cross-device communication, whereas colocating scaled operators improves utilization but risks interference across shared GPU resources like SM (Streaming Multiprocessor), memory, and interconnects. This necessitates interconnect-aware operator placement with fine-grained contention modeling.

To solve these challenges, we first perform a theoretical analysis of operator-level provisioning and scaling to characterize heterogeneity based on workload-aware sensitivity, resource elasticity, and inter-operator communication. Building upon the theoretical framework, we design OPSCALE, an end-to-end cluster-scale orchestration system across multiple planes. (1) A *data plane* captures the performance characteristics and resource sensitivity of each operator under diverse workload conditions (batch size, sequence length, query rate); (2) A *control plane* integrates these profiles with live traffic to compute optimal scaling and contention-aware placement plans; (3) An *execution plane* deploys provisioning plans, manages distributed operator lifecycles across GPU servers, and orchestrates runtime request routing. Together, OPSCALE provisions GPU resources at the operator level and scales minimally (*i.e.*, the bottleneck operators at traffic spike and the most non-sensitive operators when traffic goes down) to meet fluctuating traffic demands within the running engine.

Results. We evaluated OPSCALE on a cluster of up to 40 NVIDIA A100 GPUs and 24 GB200s, comparing against the model-level inference scaling platforms: DynamoLLM [57], AIBrix [59], and the vLLM Production Stack [20]. Across diverse model architectures (*i.e.*, text vs. vision models, dense vs. MoE models) on production-scale traces [51, 57] (processed 929K requests and 1.5B tokens), OPSCALE achieves better SLO attainment with up to 36.3% fewer GPUs and 28% lower power consumption, with gains amplified on a GB200 cluster with faster interconnects (NVL domains) [38].

Contributions. Our main contributions include:

- A systematic characterization of operator heterogeneity on compute–memory sensitivities to workload changes.
- A novel and practical operator-level provisioning strategy, to exploit intra-model heterogeneity for high efficiency.
- An end-to-end system implementation called OPSCALE that instantiates operator-level resource management.
- Comprehensive evaluations with production-scale traces to highlight OPSCALE’s superior GPU cost savings.

2 Motivation

OPSCALE targets per-model autoscaling, the dominant deployment pattern in production LLM clusters where GPUs are elastically allocated to serve each model based on its load [51, 53, 57]. A model can serve diverse workloads (*e.g.*, chats [44], deep research [46], and vibe coding [45]), each

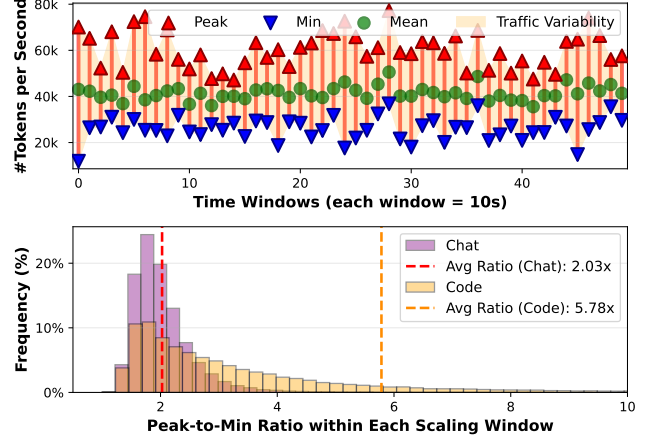


Figure 2: Burstiness in production LLM inference traffic [57] leads to large variability even within a 10-second timescale (for both Chat and Code services), making model-level autoscaling hard to adapt promptly.

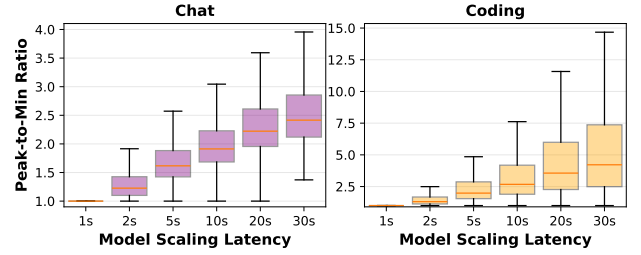


Figure 3: Larger autoscaling latency misses more fine-grained traffic variations across production workloads due to the increased load peak-to-min ratio.

with distinct traffic patterns and strict SLOs on TTFT and TBT [57]. As a result, the central challenge arises from meeting these performance targets under dynamic workloads while minimizing total resource costs.

Today, the key solution is SLO-targeted autoscaling [19, 30, 49, 57, 59]. Its goal is twofold—(1) provisioning minimal model deployment size without violating user SLOs, and (2) doing so quickly to avoid unnecessary over/under-provisioning periods that waste total GPU-hours or power.

Limitations of Model-level Scaling. Current state-of-the-art serving systems—including AIBrix [59], DynamoLLM [57], and the vLLM Production Stack [20]—rely on model-level autoscaling. This approach adjusts capacity by replicating the entire model as a monolithic unit (Figure 1, top). While straightforward to implement, this coarse-grained granularity forces a rigid coupling between model components, fundamentally hindering meeting the two requirements above.

(1) *Speed Mismatch between Model Scaling Overhead and Traffic Dynamics.* Loading a full model onto additional GPUs incurs significant startup latency (*e.g.*, loading a 70B model

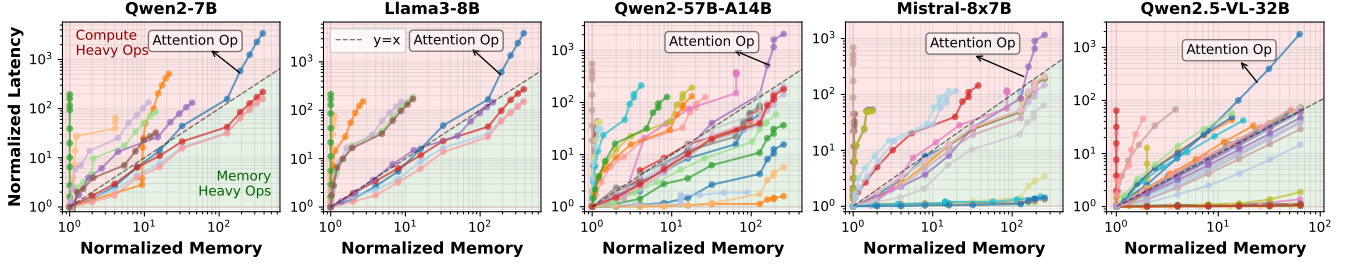


Figure 4: Operator compute and memory sensitivities to sequence lengths across different model architectures. Each line is a different operator, with each data points measured at sequence lengths in [128, 256, ..., 64K] from left to right. The $y = x$ marks the linear scaling with increased sequence length. Detailed per-operator results are provided in Figure 20 and Figure 21.

takes >10 seconds on average, even with state-of-the-art methods [16]). This delay can lag the traffic fluctuations common in LLM workloads [53, 57], often resulting in SLO violations or over-provisioning. As shown in Figure 2, production LLM traffic [57] exhibits rapid, high-amplitude bursts where peak demand can exceed the minimum by more than 2 $\times$ (for Chat) and 5 $\times$ (for Code) on average, even within a 10-second scaling window. Figure 3 further illustrates how model-level autoscaling can miss even more of these short-lived fluctuations in these workloads, as scaling latency increases.

(2) *Granularity Mismatch between Monolithic Scaling and Operator Heterogeneity.* Model-level autoscaling treats the model as a monolithic scaling unit, ignoring that LLM inference is inherently a data flow graph [69], where nodes are operators (e.g., attention, linear transformation, and normalization), and edges are data dependencies. This operator heterogeneity suggests that operators might not equally contribute to SLO violations as traffic demand increases. Consequently, monolithic scaling can waste resources by uniformly increasing capacity across all operators yet failing to precisely alleviate the truly bottlenecked operators. To examine this mismatch, §2.1 next presents a systematic characterization of operator behavior across diverse model architectures.

2.1 Operator Characterization Study

To understand how workload dynamics affect different parts of a generative model, we characterize performance metrics for each operator by varying the request sequence length, batch size, and arrival rate. We focus on metrics that can drive autoscaling decisions: compute time and memory usage¹.

Our study characterizes each operator’s per-metric *sensitivity*, quantifying the rate at which performance metrics change as workload changes. We define sensitivity as the normalized performance relative to a base configuration (e.g., batch size of one). It allows us to identify performance-critical operators. **Experiment Setup.** We consider models of two dominant architectures (Table 3): dense LLMs and mixture-of-experts (MoE). The former includes Qwen2-7B [28], Llama3-8B [25], and Qwen2.5-VL-32B [29]. The latter includes Qwen2-

MoE [27] and Mixtral-8x7B [26]. Models are served by vLLM on a server with NVIDIA A100 GPUs [37]. We employ the CUDA time profiler [41] to collect GPU kernel execution times, and instrument inference with a customized version of vLLM’s built-in `layerwise_profile` library [61]. This setup captures fine-grained kernel measurements, including compute time, weight memory, activation memory, and inter-operator communication, enabling a comprehensive analysis of runtime bottlenecks and operator-level summaries. We use CUDA Green Contexts [43] for the allocation of SM cores to each operator, and DCGM [42] for SM utilization monitoring. **Compute Sensitivity.** We profile per-operator compute time as the actual GPU execution time (μ s), recorded using CUDA event profiling. We collect the kernel-only time by excluding CPU overheads and memory-transfer latencies, as these overheads can be effectively hidden by overlapping them with GPU kernel execution using asynchronous operations.

Figure 4 (Y-axis) and Figure 5 present compute-sensitivity curves for all operators in each model, with respect to sequence length and batch size, respectively. First, we note the non-negligible spread of curves among operators within a model. It highlights how differently the workload changes impact the compute time for each operator. Second, models exhibit different degrees of heterogeneity, going from a narrower spread (e.g., Qwen2-7B and Llama3-8B in Figure 4) to a wider spread (e.g., Qwen2-57B-A14B and Mixtral-8x7B). This implies that operators do not equally contribute to SLO violations, especially in the latter case with higher diversity.

The spread of sensitivity curves arises from the intrinsic computational structure of different operators. As shown in Figure 5, operators with substantial per-token arithmetic (e.g., large MLP operations, wide linear projections, and fused MoE operations) exhibit near-linear or even super-linear scaling when batch size increases, as their arithmetic intensity dominates memory and kernel-launch overheads. In contrast, lightweight operators (e.g., layer norms, element-wise activations, small projections) scale sub-linearly because their fixed launch costs and memory-bound behavior become increasingly amortized at larger batch sizes.

Finally, attention operators exhibit the steepest growth: self-

¹Additional characterization results in Appendix C.

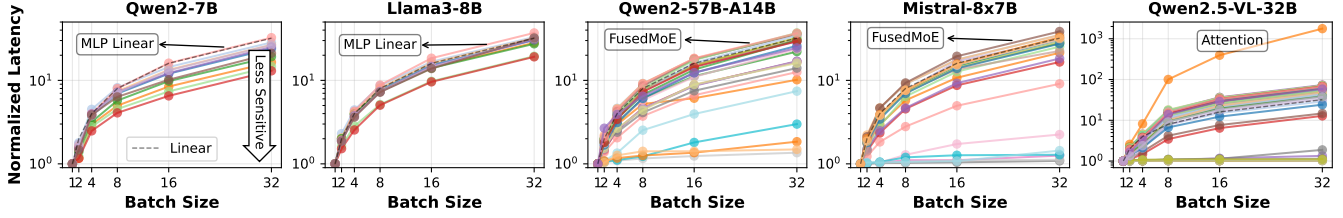


Figure 5: Operator compute sensitivity to batch sizes across different model architectures. Each line is a different operator.

attention incurs quadratic complexity in sequence length (L), while remaining linear in batch size (b). This $O(L^2b)$ complexity is significantly higher than operators that scale linearly ($O(Lb)$) with the workload, such as MLPs, norms, and embedding lookups. Therefore, linear operators (e.g., FusedMoE) can dominate compute at short sequences, and attention becomes the primary contributor as sequence length grows.

Takeaway 1: Operators exhibit varying compute sensitivity, with heterogeneity differing across models. Workload-aware, operator-level scaling of compute resources is needed rather than uniform, model-wide scaling.

Memory Sensitivity. We profile both operator weight memory and transient memory usage. The former refers to the static storage of model parameters. The latter refers to the intermediate tensors and KV cache, which are generated during inference computation. By profiling memory usage under varying sequence lengths and batch sizes, we quantify each operator’s memory sensitivity based on how its memory usage scales with these factors. Figure 4 (X-axis) shows that the attention operator dominates memory growth due to its $O(L^2)$ scaling with sequence length, while most other operators grow roughly linearly. Unlike a naïve attention implementation that materializes an $O(L^2)$ attention-score matrix, the FlashAttention implementation used in our experiments [12] avoids it and exhibits approximately linear memory growth with sequence length. Several activation-heavy kernels, such as fused act_and_mul, follow a similar trend, whereas operators such as index_select and reshape_and_cache remain nearly flat over the evaluated range of request sequence lengths.

Takeaway 2: Memory sensitivity varies less across operators and models compared to compute sensitivity. It is bounded by linear scaling with FlashAttention.

Cross-Dimension Analysis. Figure 6 plots the average per-operator sensitivity, with respect to sequence length for both compute time and memory usage, using a representative layer from the Qwen2-7B model. We observe substantial variation not only in the absolute magnitude of each sensitivity metric, but also in their relative ratio. Some operators are highly compute-intensive yet exhibit less memory growth (Attention); others are memory-intensive while remaining lightweight in compute (RMS Norm); and a few are simultane-

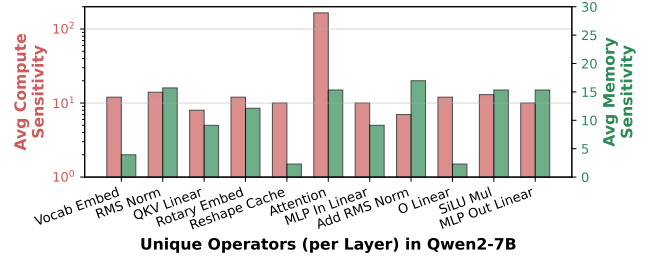


Figure 6: Averaged per-operator compute- vs. memory-sensitive in Qwen2-7B across different sequence lengths.

ously heavy or lightweight (QKV Linear) in both dimensions. This two-dimensional heterogeneity highlights that workload-induced pressure does not manifest uniformly across operators. Instead, each operator responds differently depending on the dominant resource dimension.

Takeaway 3: Operators may be sensitive in both compute and memory, in either, or even in neither. Due to such mismatch, orchestrating GPU resources requires jointly considering each operator’s sensitivity profile and this compute-memory trade-off (e.g. avoid scaling compute-cheap but memory-heavy operators to save capacity).

Sensitivity to SM Core Allocation. Modern GPUs support spatial sharing, e.g., CUDA Green Contexts [43] can specify the number of SMs allocated to each operator’s stream. To guide operator-to-device placement, we characterize the sensitivity of operator compute time to SM core allocation. We consider both the long request sequence of 2K tokens (prefill phase), and the short sequence of 1 token (decode phase).

For the long 2K-token sequence, as SM allocation increases, the normalized latency for all operators decreases significantly (Figure 7a). This observation is particularly highlighted for compute-intensive operators such as Attention and MLP. Shown in Figure 7b, the reason is that these operators naturally require many SM cores. A reduction in SM allocation results in latency increases, hence steeper sensitivity curves.

A different pattern emerges in the decode phase (1-token sequence length). Most sensitivity curves are relatively flat, with a minor latency decrease as SM cores are allocated (Figure 7a). Again, Figure 7b clarifies the reason: the SM utilization is generally too low to saturate the available resources.

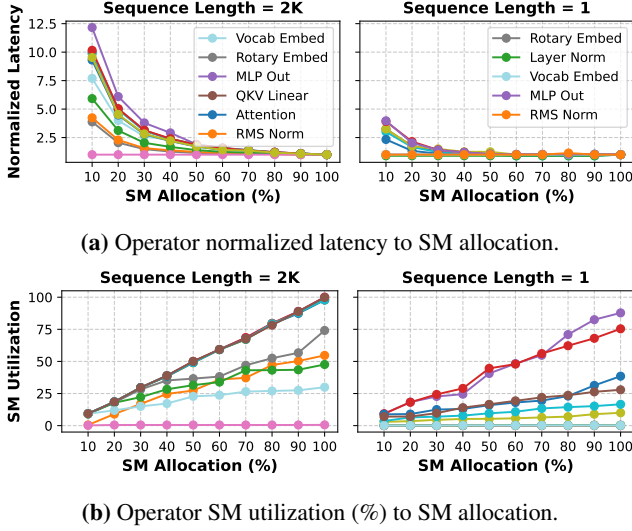


Figure 7: Performance and SM utilization characteristics across operators in Qwen2-7B under varying SM allocations.

Takeaway 4: Sensitivity to SM allocation varies widely across operators, sequence lengths, and prefill/decode phases. Particularly, spatial sharing is suitable for short sequences, as allocation reduction imposes less impact.

2.2 Rethinking the Scaling Unit

Rather than improving model-level autoscaling, the intra-layer heterogeneity (§2.1) motivates us to rethink autoscaling at a finer-grained scaling unit—the operators. Conceptually, the paradigm shift to operator-level autoscaling is feasible, as operators satisfy the desirable property of functional and temporal isolation. First, operators have explicit execution boundaries during inference. They are compiled into GPU kernels and executed within a CUDA stream, which is an ordered sequence of kernels that may overlap with data transfers to improve throughput [6]. These kernels are scheduled to run on the GPU’s SM cores. Each kernel is run-to-complete without preemption, and there are techniques allowing multiple streams to share SM cores (e.g., MPS [40], MIG [39], and CUDA Green Contexts [43]). Second, each operator maintains its weights and intermediate data in GPU memory, along with temporary buffers for transient memory needs.

3 Operator-Level Autoscaling

Building upon the insights of operator heterogeneity, a natural question arises: *what are the resource efficiency benefits expected from operator (op)-level autoscaling?* To this end, we present our analysis, based on a novel theoretical optimization problem formulation for SLO-driven op-level autoscaling. Note that solving the optimization problem results in an *offline analytical oracle* to quantify the opportunity from op-level autoscaling. The online system instead uses the millisecond-scale greedy provisioning heuristic in §4.3, whose resource

cost remains within 8% of this oracle.

3.1 Theoretical Problem Formulation

Op-level autoscaling is inherently a multi-objective optimization problem. It considers both SLO and resource objectives, and specifies the following inputs and solutions.

Problem Inputs and Solutions. LLM inference is an execution of the operator DAG [48], $\mathcal{G} = (\mathcal{V}, \mathcal{E})$: $\mathcal{V}$ is the set of operators and $\mathcal{E}$ represents data dependencies. Given a stream of requests $x \in X$, problem inputs include arrival rate λ (QPS) and request sequence length distribution ($L(x)$). Solutions are per-operator configurations, which include batch size (B_v), number of replicas (R_v), number of tensor parallelism shards per replica (P_v), and per-shard device assignment (A_v). Furthermore, for GPUs that support spatial sharing techniques such as CUDA Green Contexts [43], the configuration also specifies the number of SMs allocated to a shard (S_v).

SLO Objective. A request goes through multiple *iterations* of the operator DAG [48]. The first iteration is prefill, which processes the full input sequence $L(x)$ of the request. Each subsequent decode iteration generates one output token at a time. Two common inference serving SLOs are TTFT and TBT, for prefill and decode iterations, respectively. The latency of an iteration, T_{itr} , should be less than the SLO.

$$T_{itr} = \sum_{v \in \mathcal{V}} (T_v + C_v + W_v) < T_{SLO} \quad (1)$$

T_v and C_v are operator v ’s computation time and communication time to its downstream operators. Both depend on v ’s configurations (P_v, B_v, A_v) and request sequence length (L). W_v denotes the waiting time at each operator v modeled in networked queues [32], and each operator can be modeled as an $M/M/R_v$ queue [5] (Poisson arrivals and exponential service times). The service rate of v is $\mu_v = \frac{1}{T_v}$. Then, W_v is:

$$W_v = \frac{C(R_v, \rho_v)}{R_v \mu_v - \lambda} \quad \text{with} \quad \rho_v = \frac{\lambda}{R_v \mu_v} \quad (2)$$

where $C(R_v, \rho_v)$ is the Erlang-C formula [4].

Resource Objective. To minimize the logical shard-replica demand across all operators:

$$\min \sum_{v \in \mathcal{V}} P_v \cdot R_v \quad (3)$$

At the same time, deploying operator replicas needs to respect the memory and SM constraint of each GPU device (M_d^{cap}).

$$\sum_{v \in \mathcal{V}: A_v = d} M_v \leq M_d^{\text{cap}}, \quad \sum_{v \in \mathcal{V}: A_v = d} S_v \leq 100, \quad \forall d \in \mathcal{D} \quad (4)$$

where $M_v = M_v^{\text{weight}} + M_v^{\text{transient}}$, or weight memory and transient memory (e.g., activations, KV cache).

3.2 Benefit Analysis

Building on the analytical formulation in §3.1, we use a profiling-driven performance model that captures each operator’s compute cost, memory footprint, communication volume, and queueing behavior. $L(x)$ is derived from production LLM inference traces [57]. This model² computes the

²Accuracy of operator profiles and the analytical model is in §5.7.

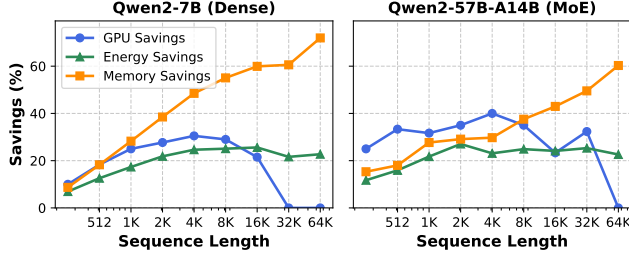


Figure 8: Benefits of op-level provisioning compared to model-level under varying sequence lengths (refer to Figure 22 in Appendix for full results).

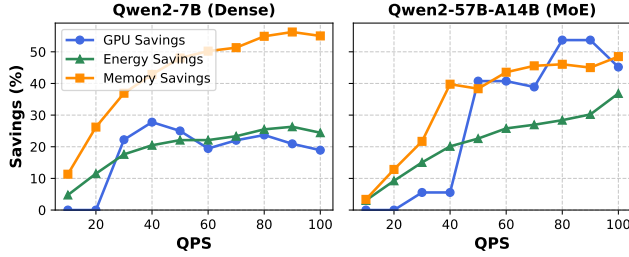


Figure 9: Benefits of op-level provisioning compared to model level under varying QPS (refer to Figure 23 in Appendix for full results).

expected TTFT, TBT, and energy consumption for any operator configuration and identifies the optimal SLO-compliant deployments by exhaustively enumerating all feasible configurations. Our analysis utilizes two representative LLMs: dense Qwen2-7B and MoE Qwen2-57B-A14B. We vary the workload from 10–100 QPS and 128–64K sequence lengths, and evaluate a spectrum of stringent to relaxed SLOs. We compare the minimal SLO-compliant provisioning plan at the model level with the optimal op-level provisioning plan using the analytical model, solved using exhaustive search.

Varying Sequence Length. Figure 8 shows that GPU savings (in blue) peak around 30% for dense model and 40% for MoE. This peak occurs at 4K-token sequences. The savings start to drop at 8K, where the compute load grows to saturate SM cores and limit GPU sharing. MoE models exhibit higher savings because their operators have a more diverse sensitivity. Energy savings (in green) follow a similar trend, reaching up to 25% reduction at peak. Unlike GPU device savings, energy savings are still noticeable beyond 4K sequences. The reason is that op-level autoscaling only needs to scale a subset of operators, in contrast to model-level autoscaling. In addition, GPU memory savings (in orange) grow steadily with sequence length, surpassing 60% at 32K sequences.

Takeaway 5: *Op-level autoscaling is most beneficial at moderate sequence lengths, though long sequences can still exhibit up to 25% energy and 60% memory savings.*

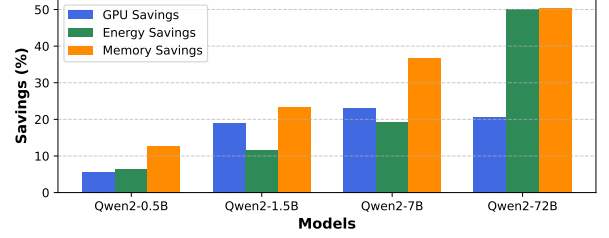


Figure 10: Saving comparisons across model sizes.

Varying QPS. Figure 9 (in blue) shows that GPU device savings peak around 30% near QPS = 40. Beyond this point, savings fluctuate slightly, as SM saturation and tighter latency margins reduce opportunities for GPU device sharing. On the other hand, savings remain negligible at low QPS (<20), where the provisioned model instance can handle limited traffic demands without scaling. Energy savings (in green) reach up to 25% at high QPS for Qwen2-7B, primarily driven by the reduced GPU device count from op-level autoscaling’s selective operator scaling decisions. Memory savings (in orange) grow steadily with QPS, surpassing 50% at 100 QPS, reflecting a similar trend to sequence length growth.

Takeaway 6: *Op-level autoscaling yields the most benefits at moderate to high QPS. The finer-grained provisioning mitigates queueing delays, without excessive allocations.*

Large vs. Small Models. We analyze the factor of model size using the Qwen2 model family, ranging from 0.5 to 72 billion parameters. As shown in Figure 10, op-level scaling yields substantial savings, especially as the model size scales up. For smaller models, savings are generally modest. The reason is that operators do not exhibit a high degree of heterogeneity, which necessitates going finer than model-level scaling. However, as model size increases to Qwen2-1.5B and Qwen2-7B, GPU and energy savings rise to ~20–30%, while memory savings exceed 35%. For the largest model, Qwen2-72B, both energy and memory savings peak at ~50%. Large models tend to exhibit a higher degree of operator heterogeneity, encouraging the use of finer-grained resource provisioning³.

Takeaway 7: *Larger models benefit more from op-level autoscaling, due to greater operator heterogeneity.*

4 OPSCALE Design and Implementation

Realizing op-level scaling in practice requires overcoming significant profiling, optimization, and runtime challenges at such fine granularity. Specifically, the finer granularity results in a growing number of scaling units. To this end, we design a system called OPSCALE, to tame this resulting complexity with two decoupled planes (Figure 11).

³Additional analysis results on op-level provisioning (including a study on prefill and decode stages) are in Appendix E.

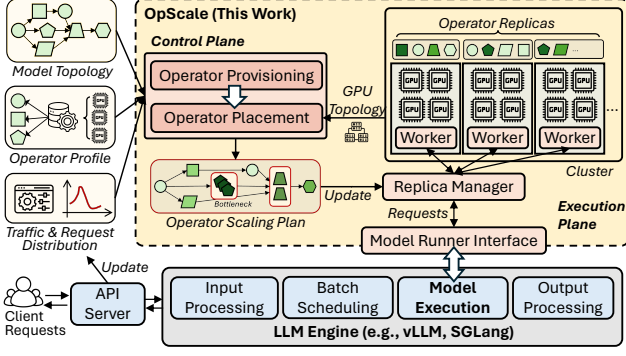


Figure 11: OPSCALE architecture overview.

Control Plane. As the runtime core of OPSCALE, the control plane operates asynchronously to compute an *operator scaling plan* (specified in §3.1) through three modules:

1. **Operator Profiler** (§4.2): An offline module that profiles three operator attributes: (1) computation time (T_v), (2) memory usage (M_v), and (3) communication overhead (C_v) from its topological dependencies. Profiling is a one-time effort and can be reused across model architectures.
2. **Operator Provisioning** (§4.3): A module that computes optimal operator configurations (P_v, R_v, B_v) based on operator profiles, current traffic, and model topology.
3. **Operator Placement** (§4.4): A module that maps logical replicas to physical GPUs (A_v, S_v), accounting for resource contention and cross-device communication.

Execution Plane. A centralized *Replica Manager* (§4.5) orchestrates replica lifecycles and serves as a request dispatcher. Unlike model-level routing, which assigns each request to a full model replica, OPSCALE routes requests through pools of operator replicas: each stage gathers outputs from its replica pool and forwards the activations to the next stage’s operators. This routing employs a weighted shortest-queue policy, accounting for SM allocation and colocation contention.

4.1 Complexity and Tractability

The complexity from a growing number of scaling units is reflected in the combinatorial explosion of operator profiling and configuration spaces. While §3.1 provides the theoretical basis for optimal op-level autoscaling, applying it directly to a runtime system can be impractical, especially with the prohibitive cost of running exhaustive search. We discuss the two main space explosion problems next.

Operator Profiling Space. Characterizing operator sensitivities ideally requires profiling operator behavior (T_v, M_v, C_v) across a vast state space, which is defined by batch size (B), sequence length (L), and SM allocation percentage (S). The problem arises as we consider their ranges. A typical setup has $B \in [1, 256]$, $L \in [1, 65536]$ (discretized), and SM allocation percentage $S \in [1, 100]$. Here, a naive exhaustive search would require profiling over 10^7 configurations *per operator*. Even if each micro-benchmark run takes only 100 ms, exhaustively

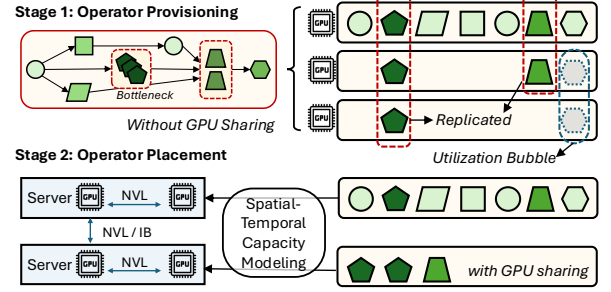


Figure 12: OPSCALE control plane consists of operator provisioning (§4.3) and placement (§4.4).

profiling a 0.5B-parameter model (with 11 unique operators) would require *weeks* of GPU time. This is untenable for model iteration and deployment in production.

Operator Configuration Space. Deriving the optimal operator scaling plan is an NP-hard integer non-linear programming problem (§3.1). The search space grows combinatorially with the number of operators and each of the configuration dimensions (P_v, R_v, S_v, A_v). Brute-force solvers take *minutes* for even a small 0.5B-parameter model, while production traffic requires high-quality decisions within seconds (Figure 2).

4.2 Operator Profiling

Our experience with exhaustive profiling highlights an observation, where it ignores patterns in operator behavior. Considering compute sensitivity, while Figure 5 shows that operators can have seemingly different sensitivity curves with respect to batch size changes, they mostly exhibit a monotonic trend.

The observation above suggests that it is feasible for Operator Profiler to employ a *sparse sampling* strategy. Instead of performing exhaustive enumeration, we first sample random points in the (B, L) space. These points are then used by techniques such as piecewise interpolation. Interpolation is simple, as it is non-parametric and does not require training prior to estimating an unsampled point.

Furthermore, we exploit the *structural equivalence* of LLM architectures. As some operators (e.g., Attention and MLP) are identical across layers and models, their profiles can be reused. With parallel execution of sampling tasks, multi-billion-parameter models can be profiled in under an hour.

4.3 Operator Provisioning

To navigate the combinatorial explosion challenge, OPSCALE leverages an observation derived from operator heterogeneity. Specifically, sensitivity curves suggest that operators experience different degrees of performance impact due to workload changes. In other words, by prioritizing critical operators (*i.e.*, operators that expect the most impact) for provisioning first, we avoid exhaustively provisioning *all* operators in the model.

This optimization process proceeds in two distinct phases⁴:

⁴Algorithm 1 in Appendix B.

1. **Initialization (Local Optimality):** We first establish a baseline configuration by inheriting the initial parallelism P_v from the model deployment strategy (*i.e.*, tensor parallelism for intra-server and pipeline parallelism for cross-server deployment [60]). For each operator, we scan feasible batch sizes $b \in \{1, \dots, B_v^{\max}\}$ to find the minimal replica count $R_v(b) = \lceil \lambda / \mu_v(b, P_v) \rceil$ required to sustain the request arrival rate with no queue build-up (Eq. 2). We select the (B_v, R_v) pair that minimizes the local sojourn time $T_v + W_v$, providing a latency-optimal starting point.
2. **Iterative Critical Path Optimization:** The algorithm then evaluates the global DAG latency T_{total} against the SLO by iteratively identifying the current critical path:
 - **Upscaling (SLO Violation):** If $T_{\text{total}} > \text{SLO}$, we identify the bottleneck operator v that contributes the most to the critical path latency in the DAG. We greedily apply the specific adjustment: either increasing replicas R_v or tuning (B_v, P_v) that yields the maximal latency reduction per unit of resource cost. Repeat until SLO is met.
 - **Downscaling (Resource Reclamation):** Conversely, if $T_{\text{total}} < \text{SLO}$, we relax the configuration of the top non-critical operators to reclaim resources, ensuring the system operates at the efficiency Pareto frontier.

Unlike the exhaustive search used only for the offline benefit analysis (§3.2), this greedy method runs online and achieves near-optimal resource efficiency: its resource cost is within 8% of the brute-force oracle, while online incremental plan generation takes only milliseconds (§5.6).

4.4 Operator Placement

OPSCALE treats placement as a multi-dimensional bin-packing problem, mapping operator replicas onto minimal GPUs. For GPUs supporting spatial sharing, OPSCALE can further reduce the total GPU footprint, by colocating operators. However, this requires the placement algorithm to be aware of the colocation interference. This section first discusses our interference model, and then presents a locality-aware *Best-Fit Decreasing* heuristic with spatial-temporal modeling⁵.

Interference Model. As operators share a device via CUDA Green Contexts, contention on HBM bandwidth, L2 cache, and SM schedulers introduces performance interference. Let operator replica v run on device d with batch size b and SM fraction p ; we define an *interference factor*:

$$I_{d,v}(b, p) = \tilde{T}_v(b, p) / T_v(b, p) \geq 1, \quad (5)$$

where $\tilde{T}_v(b, p)$ is the co-execution latency and $T_v(b, p)$ is the isolated latency at the same SM allocation. This formulation captures co-location effects in a single empirical factor $I_{d,v}(b, p)$, which is used by Alg. 2 without additional modeling assumptions. During the one-time offline profiling effort, we sweep sequence length and SM splits at 5% granularity for each operator-type pair, co-running both operators on separate green-ctx streams. The resulting $I_{d,v}$ values are stored

in a lookup table; at placement time, queries take $O(1)$ time, with configurations snapped to the nearest profiled grid point. Our model covers operator *pairs*, approximating three-way co-location by multiplying pairwise factors. We validate prediction accuracy against held-out measurements in §5.7.

Best-Fit Placement Algorithm. With $I_{d,v}$ defined, the placement heuristic proceeds in three steps:

1. **Decomposition & Ordering:** To simplify the packing problem, we first decompose the global resource demand into two sets: (1) **Base Instances** ($\mathcal{R}_{\text{base}}$), the minimum full-model replicas required for baseline service stability, deployed monolithically; and (2) **Elastic Components** ($\mathcal{R}_{\text{extra}}$), the remaining unassigned operator replicas to be packed dynamically. We then sort $\mathcal{R}_{\text{extra}}$ in descending order of computational cost T_v (or dominant resource demand). This prioritizes bottleneck operators when the cluster has the most contiguous space, reducing “tail fragmentation” where only small gaps remain for heavy operators.
2. **Greedy Best-Fit Placement:** For each replica r , we search for a candidate device $d \in \mathcal{D}$ that minimizes the *residual capacity vector* $\|\mathbf{C}_d - \mathbf{v}_r\|$, where $\mathbf{C}_d$ denotes the current available resources on device d . A placement is considered valid only if it satisfies the safety constraint. Using the interference model (Eq. 5), we compute the interference-adjusted latency $T'_v = T_v \cdot I_{d,v}(b_v, p_v)$ and verify that the resulting global latency remains within the model’s SLO. Only when no existing device can accommodate r , a new device is activated, thereby minimizing active GPUs.
3. **Locality-Aware Tie-Breaking:** When multiple devices provide comparable fits, OPSCALE prioritizes placements that preserve data locality, minimizing communication overhead. It further ranks candidates by communication hierarchy, favoring intra-device, then intra-server (NVLink) or NVL domain, and cross-server (InfiniBand) placements.

The same placement routine is applied incrementally during autoscaling events. Placement is not accepted independently of provisioning: for every candidate device, OPSCALE feeds the profiled communication cost and interference-adjusted operator latency back into the end-to-end latency model. A candidate that would violate the SLO is rejected; if no existing device remains feasible, OPSCALE activates a new device. Thus, provisioning proposes the logical replica configuration, while placement verifies that the physical realization preserves its SLO estimate. By avoiding global re-optimization, this greedy strategy maintains low decision latency while providing high-quality placements in dynamic traffic conditions.

4.5 Execution Plane

OPSCALE’s *Execution Plane* actuates the scaling plan generated by the control plane. To support efficient op-level replicating and execution, we extend the LLM engine runtime with the following novel features and optimizations.

Dynamic Operator Replicating and Registering. Existing LLM engines assume a static execution graph with fixed op-

⁵Algorithm 2 in Appendix B.

erators, whereas OPSCALE must dynamically adjust operator replicas and execution paths. To support this, OPSCALE introduces a *Replica Manager* that maintains a dynamic registry of *Operator Replicas*, automatically registering new replicas. A late-binding mechanism via `forward_pre_hooks` transparently intercepts operator calls, to select the appropriate replica at runtime. This enables in-flight requests to immediately switch to the new execution graph with zero downtime.

Moreover, replicating operators requires allocating GPU memory at runtime while existing LLM engines require all memory to be pre-allocated during engine initialization. To solve this problem, OPSCALE implements an *ElasticBlock-Manager* by integrating kvcached [66, 67]. This allows the engine to dynamically allocate memory for newly created replicas and reclaim memory from those removed.

Multi-Stream Management. For a model with D layers, $|\mathcal{V}|$ operators per layer, and up to R replicas per operator, naively creating a unique stream for every instance incurs prohibitive performance overhead due to the frequent switching between streams ($O(D \times |\mathcal{V}| \times R)$). We address this bottleneck via a two-tier strategy: (1) *Stream Reuse* exploits the sequential nature of layers. A `GlobalStreamPool` recycles streams across pipeline stages. This bounds resource overhead to $O(R)$ regardless of model depth. (2) *Cross-Device Pipelining* hides the communication latency between consecutive operators. A `MultiDeviceStreamPool` pipelines remote data transfers (e.g., NVLink) with kernel execution, ensuring data arrives exactly when the compute kernel is ready. Each stream is further bound to a green-context resource, with SM allocation derived from the interference model (Eq. 5). This enforces hardware-level partitioning at dispatch time.

Operator Pipelining and Dispatching. Monolithic execution runs all operators on a single CUDA stream, creating utilization bubbles as later operators must wait for earlier ones. To improve throughput, inspired by Nanoflow [69], OPSCALE adopts pipelined execution, allowing operators to run concurrently upon available inputs. Requests are dispatched to operator replicas via shortest-queue routing, weighted by replica capacity, and dependent data is synchronized between operators using lightweight CUDA events.

5 Evaluation

We prototyped OPSCALE on top of nano-vLLM [18] in $\sim 17K$ lines of Python code. nano-vLLM is a minimalist inference framework that inherits key optimizations from vLLM (e.g., PagedAttention, continuous batching, and tensor parallelism), ensuring that our results are representative of modern high-performance LLM serving systems. For a controlled comparison, *all* evaluated approaches use the same nano-vLLM inference engine, including the same model implementation, kernels, batching scheduler, KV-cache manager, and tensor-parallel backend. The experiments reproduce each baseline’s autoscaling policy over this common data plane; the intended differences are the scaling policy and granularity, rather than

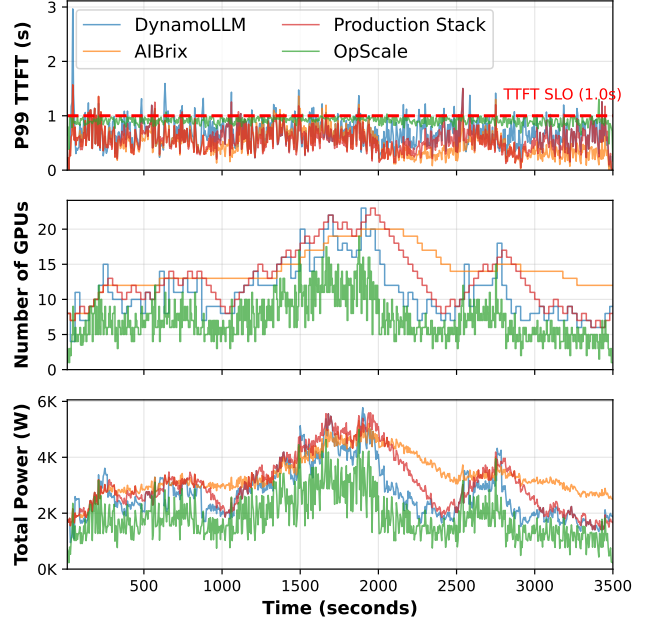


Figure 13: Comparison of request performance, GPU usage, and cluster power usage during autoscaling for Qwen2-7B.

the underlying inference runtime.

5.1 Experiment Setup

Models and Traces. We evaluate OPSCALE using two representative LLMs: Qwen2-7B [28] (a dense LLM) and Qwen2-57B-A14B [27] (an MoE LLM). These models capture two widely used architectures in generative models. We use production LLM serving traces [57] which include temporal request arrival patterns and sequence lengths collected from real-world interactions, enabling controlled and reproducible evaluations of autoscaling under dynamic load conditions.

Hardware. Most experiments are conducted on a 40-GPU cluster of five Azure GPU VMs [37]. Each VM is equipped with 8 NVIDIA A100-80GB GPUs connected via NVLink and high-speed InfiniBand networking. For sensitivity study on hardware and granularity, we also evaluate OPSCALE on a cluster of 24 GB200 GPUs within the same NVLink domain.

Baselines. We compare OPSCALE against state-of-the-art *model-level* autoscaling systems ported over the common nano-vLLM backend: (1) DynamoLLM [57] (SLO-driven), which dynamically adjusts model replicas based on max per-instance serving throughput under given TTFT and TBT SLOs. (2) AIBrix [59] (utilization mode), which uses GPU utilization metrics to decide scaling actions. (3) Production Stack [20] (queue-driven), which triggers scaling based on the number of pending prompt tokens in the queue. We adopt the recommended configurations in each baseline’s documentation. Model scaling frequency is set to every 20 seconds. Additionally, we add (4) DVFS [57] and μ -Serve [54] for power consumption comparison, and (5) Attn-FFN disaggre-

gation [62] for scaling granularity study.

5.2 Dynamic Autoscaling

We first evaluate OPSCALE and baselines under dynamic load patterns to assess autoscaling behavior, including GPU usage, request SLO attainment, and total power consumption. In total, each model serves 929K requests (1.5B prompt tokens) replayed from the production traces [57].

Figure 13 presents the autoscaling comparison in a one-hour sample of the trace for Qwen2-7B (MoE results are in Appendix F). OPSCALE consistently provisions fewer GPUs while delivering the highest SLO attainment across both dense and MoE models: OPSCALE requires on average 7.1 GPUs, a 35–50% reduction compared to DynamoLLM (11.2), AIBrix (14.3), and Production Stack (13.0). This is because (1) OPSCALE only scales out critical operators, and (2) it scales down faster to avoid idle provisioning. Consequently, we observe a reduced power usage (bottom row) from OPSCALE, even though the per-GPU utilization is higher. In addition, OPSCALE’s faster elasticity (1 second) and fine-grained optimization loop (§4.3) help achieve the best SLO attainment: 98.4%, compared to 88–95% for the baselines.

For the MoE model (Figure 27), the trend is similar, but with even higher GPU savings. This aligns with the theoretical analysis (§3.2). OPSCALE allocates only 10.8 GPUs on average, much lower than DynamoLLM (17.3), the GPU-util-based autoscaler (23.1), and the queue-based baseline (18.0). SLO attainment improves correspondingly: OPSCALE reaches 98.1%, compared to 84.2–97% for baselines. These differences match the behavior visible in the figure — baselines frequently over-allocate GPUs yet still incur noticeable tail TTFT spikes during fast load surges, whereas OPSCALE maintains stable latency with a substantially smaller footprint.

Where Do the Savings Come From? The savings come from two sources. First, OPSCALE scales at the operator level rather than at the model level. The bottleneck operators shift continuously with the workload. At short sequence lengths, linear project and MLP operators can dominate compute. As sequence length grows, attention becomes increasingly critical because of its steeper sequence-length sensitivity. OPSCALE tracks this shifting profile and redistributes resources across the full operator set, avoiding the waste of scaling operators that are not the current bottleneck. Second, scaling itself is fast: sub-second operator actuation keeps the actuation portion of the response to transient spikes small, avoiding the standing overprovisioning that would otherwise be needed to absorb demand while slower scaling catches up.

Impact on Power Consumption. Figure 13 shows that OPSCALE consistently consumes the lowest cluster-level power (mainly by reducing operator replicas and GPU idle power).

To further evaluate power–performance trade-offs, we compare OPSCALE against two *power-aware* baselines: (1) DynamoLLM+DVFS (enabling GPU frequency scaling) and (2) μ -Serve++ (augmenting μ -Serve’s power-aware model

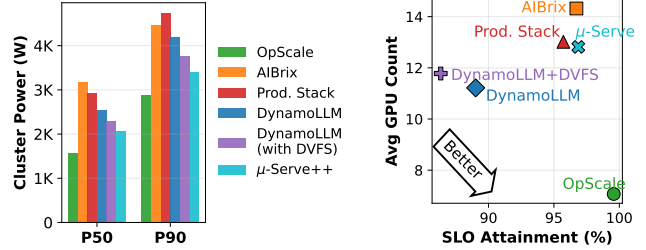


Figure 14: Cluster-level power consumption comparison.

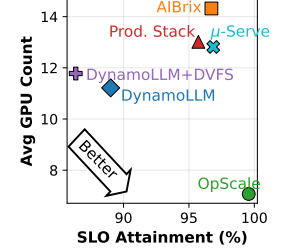


Figure 15: Cost vs. SLO attainment trade-off.

Table 1: Scale-up latency on Qwen2-7B.

Approach	P99 latency	P90 latency	Avg. latency
Model-level	11.55 s	11.04 s	10.68 s
OpScale (1 op)	0.10 s	0.05 s	0.03 s
OpScale (50% ops)	0.42 s	0.26 s	0.18 s
OpScale (all ops)	0.45 s	0.38 s	0.33 s

partitioning and placement with DynamoLLM’s autoscaling logic). Figure 14 shows that OPSCALE achieves the lowest power consumption at both P50 and P90 (20% and 16% less than the best power-aware baseline μ -Serve++). Figure 15 further illustrates the trade-off between performance and cost. While DVFS-based baselines like DynamoLLM+DVFS and μ -Serve++ improve power consumption, they lag behind OPSCALE in both dimensions of SLO attainment and GPU costs. These results confirm that structural op-level elasticity is more effective for system-wide power savings than coarse-grained replica scaling or DVFS. MoE results are in Appendix F.

Scale-up Latency. A major contribution to better SLO attainment under dynamic traffic is the rapid response to the demand changes. To understand elasticity limits, we compare the scaling overhead of op-level provisioning against traditional model-level scaling across a range of scaling intensities. Table 1 reports results on Qwen2-7B. Model-level scaling incurs non-trivial per-model-instance latency of 10.7 seconds on average, as each scale-out event requires (1) loading the full set of model weights and (2) provisioning an additional LLM engine control plane. On the other hand, OPSCALE needs to load and initialize replicas only for the provisioned operators. In addition, it avoids replicating the LLM engine control plane, which together reduces scaling overhead by up to two orders of magnitude (P99 <0.45 s), even when provisioning many operators simultaneously. A fully warm standby instance can reduce this startup latency, but it must reserve enough GPU memory and compute capacity for an entire model replica even while idle. Under our cost objective, such reserved GPUs are counted as provisioned capacity; a warm standby therefore exchanges persistent over-provisioning for faster response rather than eliminating the scaling cost. Table 1 instead compares on-demand scale-out without pre-reserving

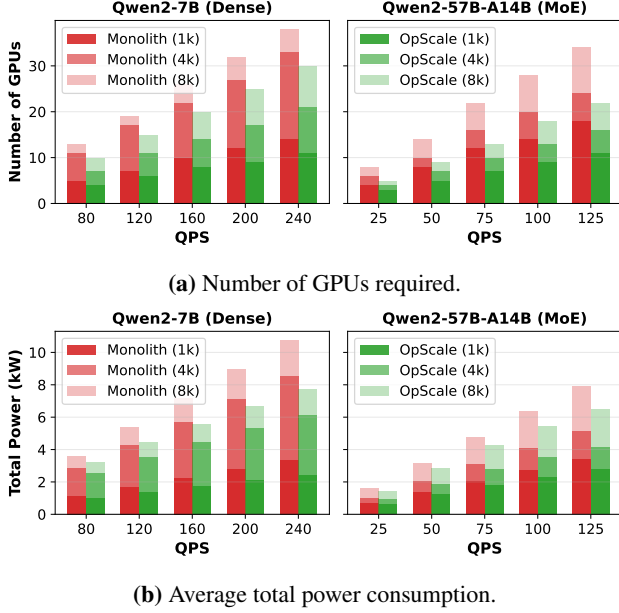


Figure 16: Comparison of the serving cost (in total GPUs and total power consumption) for varying QPS.

an additional full-model replica.

5.3 Cost Savings when Meeting SLOs

To understand the limits of cost savings, we evaluate the minimum capacity required to sustain varying workloads, while meeting the same latency SLOs. For each QPS level, we configure and scale both systems (model-level vs. OPSCALE) until the SLO violation rate falls below the threshold (*e.g.*, when P99 TTFT of Qwen2-7B is smaller than one second).

Figure 16 presents the provisioned capacity, measured in total GPUs and total power draw, required to sustain increasing traffic in QPS. Across both dense and MoE models, OPSCALE consistently achieves the target SLO with 20.1% and 35.7% fewer GPUs on average, respectively, for requests with 1K sequence lengths. This reduction stems from OPSCALE’s fine-grained op-level provisioning, which avoids the over-provisioning required by monolithic model-level provisioning. OPSCALE’s savings for longer requests vary from 36.3% (for 4K) to 22.1% (for 8K) under the same SLO target, aligning with our observation from the theoretical analysis (§3.2).

The reduction in GPU usage directly translates into lower power consumption (Figure 16b). At high load, OPSCALE reduces cluster-wide power draw by 14–28% (depending on the model and sequence length), even though the per-GPU power can be higher in OPSCALE (*e.g.*, 282 W vs. 245 W on average). Notably, unlike baselines whose power usage grows sharply with QPS due to coarse scaling actions in rigid GPU increments, OPSCALE maintains a smoother growth curve, reflecting its ability to exploit latent compute slack at the operator level before allocating additional hardware.

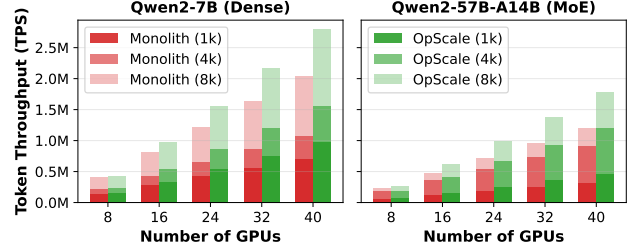


Figure 17: Max throughput achieved at fixed provisioning.

5.4 Throughput Improvements

Op-level provisioning also benefits *static* deployments, with a fixed number of GPUs. In static clusters up to 40 GPUs, we increase the request arrival load until each system reaches its SLO limit and report the maximum sustainable input token throughput (TPS). Shown in Figure 17, OPSCALE achieves higher throughput from the same hardware allocation. For the dense Qwen2-7B model, OPSCALE improves peak TPS by 3–38% across cluster sizes and request sequence lengths. The improvement is even larger for the MoE model, where OPSCALE achieves up to 44% higher TPS at 40 GPUs. These improvements follow the same mechanism as the dynamic scaling results: the fixed budget is directed toward the operators that limit throughput. At shorter contexts this favors the heavy linear or fused expert operators, while longer contexts shift capacity toward attention; lightweight operators retain fewer replicas and are packed into the residual SM/memory capacity. Model-level provisioning cannot make this exchange because each added replica duplicates every operator.

Beyond Attn-FFN. We further compare OPSCALE against Attn-FFN partitioning [62, 70, 71], which only decouples attention and FFN blocks (Figure 18). While Attn-FFN reduces GPU demand compared to model-level baseline, it is consistently outperformed by OPSCALE’s full operator-level elasticity. On A100 clusters, OPSCALE reduces GPU demand by up to 33%, translating to a $1.7\times$ throughput gain. The advantage of OPSCALE scales with hardware interconnect. When switching from A100 cluster to GB200 [38] (with NVL domain), OPSCALE reduces the required GPU count by 52% at peak load while the benefits drop to 38% for the baseline. With op-level decoupled provisioning, OPSCALE better exploits high-speed interconnects to avoid the resource fragmentation inherent in coarser block-level or replica-level approaches.

Overall, even without elastic scaling, OPSCALE can drive significantly higher throughput from the same provisioned devices, demonstrating improved efficiency under *both* dynamic (§5.2) and static (§5.4) deployments.

5.5 Robustness and Sensitivity Analysis

We analyze how OPSCALE behaves under different autoscaling intervals and SLO targets, to characterize its robustness. Shown in Figure 19 (left), the SLO attainment rate increases

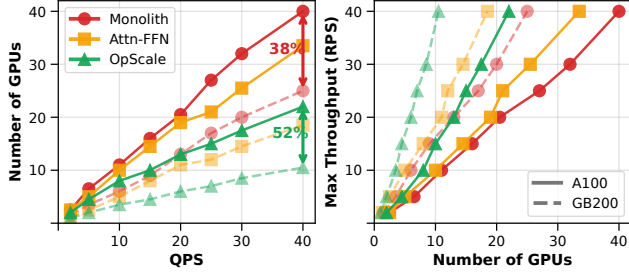


Figure 18: Static provisioning comparison across serving granularities and hardware configurations.

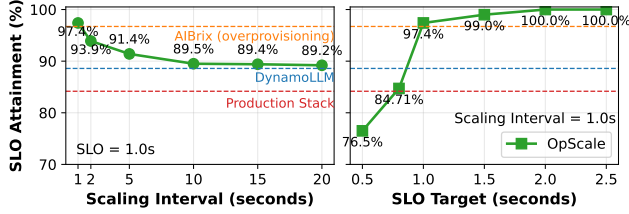


Figure 19: (Left) Impact of the scaling interval on SLO attainment. (Right) Impact of SLO target. Both with Qwen2-7B.

as the autoscaling interval decreases within the 1–20 seconds range, reaching 97.4% when the interval is set to one second. This aligns with our observation from serving production LLM workloads (Figure 3), where a shorter scaling interval allows the system to react before the traffic deviates significantly, reducing queue buildup and improving SLO attainment. Horizontal lines indicate model-level approaches scaling every 20 seconds. At a 20-second interval, OPSCALE’s SLO attainment drops to 89%, still surpassing the model-level baselines. The only exception is AIBrix, which unfortunately overprovisions with the utilization-based scaling mode.

In Figure 19 (right), we fix the autoscaling interval to one second and look at the SLO attainment rates under varying SLO targets. SLO target for the baselines (dashed lines) is one second. OPSCALE achieves 100% SLO attainment when the SLO target is relaxed, but the benefits of op-level autoscaling would diminish. On the other hand, even with a stricter SLO (0.8 seconds), OPSCALE achieves the same SLO attainment, compared to Production Stack with an SLO of one second.

Additional sensitivity analysis on model sizes, prefill/decode stages, sequence lengths are in §3.2 and Appendix E.

5.6 OPSCALE Overhead

Online Overhead and Scalability. OPSCALE imposes negligible control-plane overhead. For Qwen2-7B, the optimizer computes a full scaling plan in 2.6 ms median (P99 3.2 ms), including 2.5 ms for provisioning and 0.1 ms for placement, all on CPU. This provides $\sim 370\times$ headroom in a 1-sec scaling interval. For the larger Qwen2-57B-A14B model, total time remains low at 4.4 ms. At the execution plane, overhead consists of (1) per-operator load-balancing lookups and (2)

Table 2: Accuracy of profiling-driven models (relative error).

Component	Error Rate Metric	Average	P90
Operator profile	Sensitivity prediction	7%	15%
SM contention model	Slowdown prediction	5%	9.4%
Queueing model	Latency prediction	0.8%	1.9%

per-replica dispatch costs. This contributes a 0.3% total overhead (*i.e.*, ~ 1.5 ms out of a ~ 500 -ms prefill iteration). We mitigate dispatch overhead by replacing blocking synchronizations with event-based ordering. With lazy-sync optimization and pipelining the remaining $90\mu\text{s}$ of CPU work behind GPU kernel execution, OPSCALE successfully hides architectural overhead, ensuring total overhead in the execution plane remains negligible relative to end-to-end performance.

Offline Profiling Overhead. Operator profiling is a one-time, offline effort. With sparse sampling and structural equivalence across layers (§4.2), profiling a 57B model can complete in under an hour, on a single GB200 node. These costs are amortized across model generations; because successive releases within a model family typically retain core operator types (*e.g.*, Attention and MLP with identical kernel implementations), profiles collected for one generation largely transfer to the next—only operators whose kernel structure has changed (*e.g.*, a switch from multi-head to grouped-query attention) require re-profiling. For example, the Llama 2 $\rightarrow$ 3 and Qwen 1.5 $\rightarrow$ 2 transitions preserve the same SwiGLU MLP and RoPE attention operators; only the attention head grouping and vocabulary-embedding dimensions differ, limiting re-profiling to a small subset of operators. Cross-family reuse is also feasible when models share common operator implementations (*e.g.*, FlashAttention), further reducing the marginal profiling cost for new models.

5.7 Profiling-driven Model Accuracy

We evaluate the accuracy of the underlying models that drive OPSCALE’s provisioning and placement decisions. Summary results are reported in Table 2.

Operator Sensitivity Modeling. We compare predicted performance from our piecewise sensitivity model (§4.2) against measured runtime metrics. The average relative error is 7% (P90 15%). Since operator provisioning depends primarily on identifying *relative bottlenecks* rather than absolute latency values, this fidelity is sufficient to guide optimal scaling while significantly reducing profiling effort.

SM Contention Modeling. OPSCALE models SM contention when operators are colocated on the same GPU. Unlike prior interference models [9, 13, 55] that struggle to generalize [24], OPSCALE’s empirical degradation model (§4.4) estimates latency inflation as a function of allocated SM fractions. During runtime validation, the average slowdown prediction error is 5% (P90 9.4%). This accuracy enables effective colocation and achieves substantial throughput gains (§5.4) by balancing spatial and temporal GPU resources. While our approach

operates at the SM allocation granularity, incorporating finer-grained kernel-level spatial and temporal multiplexing (*e.g.*, LithOS [11]) could further reduce interference and improve accuracy, which we leave to future work.

Queue Modeling. The analytical networked queueing model underlying the control plane (§3) estimates the end-to-end iteration latency by composing per-operator sojourn times. Across all evaluated scenarios and bursty trace replays, the average relative error is 0.8% (P90 1.9%). This high fidelity demonstrates that the queueing network abstraction of operators is robust to realistic workload variability.

6 Discussion

When to Adopt Op-Level Autoscaling? Op-level autoscaling is most beneficial when workloads do not require *ultra-low* latency (which is common, and growing agentic applications have even more relaxed SLOs [47]) and models continue to exhibit heterogeneous operator behaviors (as shown in §2.1). These two factors present opportunities of model deployment flexibility, in order to achieve resource efficiency. In contrast, inference systems pursuing ultra-low latency through megakernel [3, 8] (fusing all operators into a single large kernel) leave limited room for op-level provisioning and scaling.

Prefill-Decode Disaggregation or Co-location? Our analysis (§E) shows that prefill stages, being compute-intensive and variable in demand, gain the most from fine-grained autoscaling. Decode, while often constrained by tighter latency budgets, benefits less from op-level provisioning. This suggests that enabling prefill-decode disaggregation complements op-level scaling by exposing varying extents of optimization opportunities across stages. In prefill-decode co-location cases with chunked prefill [1, 2], the maximum sequence length is bounded by the chunk size, altering operator load distribution. Op-level autoscaling remains valuable in such cases, as it can adapt provisioning to dynamic chunking [2].

Op-level Resharding. Although OPSCALE supports dynamic op-level resharding (changing tensor parallelism) online (§3.1), we observe that horizontal scaling (*e.g.*, scaling replicas) consistently delivers higher throughput-per-GPU gains across traces. In addition, due to weight redistribution and synchronization costs across GPUs, op-level resharding incurs $11\times$ higher overhead (P99 1.15s) than replica scaling.

Multi-Tenant, Multi-Model Serving. OPSCALE focuses on single-model serving, where a dedicated set of GPUs is provisioned for one model—the predominant deployment pattern in production LLM clusters [51, 53, 57]. However, it is complementary to multi-model and multi-tenant GPU sharing [15, 34, 50, 65–67]. First, as OPSCALE manages individual operators rather than monolithic model replicas, it can pack operators from *different* models onto shared GPUs with its contention-aware placement algorithm (§4.4). Second, scaling down non-critical operators of one model frees SM/memory that can be immediately reused by operators of another model. This degree of reuse is beyond model-level scaling. Extending

to multi-tenant settings with request-level multiplexing across models requires tenant fairness policies and cross-model interference modeling, which we leave to future work.

Autoscaling. Autoscaling has been a crucial technique for system operations, even before the emergence of LLM serving. One system with such extensive efforts is microservices [17, 52, 63, 64]. While the overall concept of autoscaling remains, LLMs require us to revisit prior assumptions. For example, while CPU usage alone is a strong signal for autoscaling microservices in production [33, 64], LLM inference is strongly sensitive to workload characteristics and GPU resources. To this end, OPSCALE takes the first step to formal characterization of LLM inference at op-level granularity, and addresses provisioning and scaling challenges in practice.

7 Related Work

Disaggregation in LLM Serving. There has been a trend of disaggregation on generative model serving: (1) Prefill-Decode disaggregation [48, 68], enabling independent autoscaling of each stage. (2) Encoder–LLM disaggregation [14, 53, 56] and VAE-DiT disaggregation [23] focus on modality-specific scaling. (3) MA parallelism [10, 36, 62, 70, 71] disaggregates attention and expert/FFN layers to leverage pipelining and improve throughput. OPSCALE differs from these fixed decompositions in both objective and mechanism: it identifies workload-dependent bottleneck operators, independently adjusts their capacity, and jointly places the resulting replicas under communication and interference constraints. Thus, the contribution is not decomposition alone, but using operator-level elasticity as a cluster-wide autoscaling and placement mechanism.

LLM Autoscaling Policy. As mentioned in §2, AIBrix [59], DynamoLLM [57], Chiron [49], DeepServe [22], and SageServe [30] propose model-level autoscaling *policies* that combine demand prediction with replica management. Our work is orthogonal: rather than policy design, we contribute a new autoscaling *mechanism* at the operator level, which benefits from advanced autoscaling policies in this field.

Multi-Stream Scheduling. Recent work [7, 21, 31, 35, 69] has leveraged multi-stream processing to improve utilization and throughput, distinct from our focus on autoscaling and fine-grained resource management. Nanoflow [69] exploits intra-device parallelism by overlapping computation with I/O, while Pod-Attention [31] collocates prefill and decode stages to saturate compute and memory bandwidth. Bullet [35] further utilizes multi-stream processing for higher utilization. These approaches focus on optimizing the *execution pipeline* of a single model instance. OPSCALE is orthogonal to these techniques by exploring the optimal serving granularity and how each operator should be provisioned and dynamically scaled across the GPU cluster to minimize serving costs.

Resource-Level Multiplexing. Several systems optimize model execution by fine-grained decomposition for better resource utilization. Orion [58] and LithOS [11] focus on

kernel-level spatial and temporal multiplexing to improve intra-GPU utilization in *multi-tenant* environments. Similarly, μ -Serve [54] exploits operator sensitivity to GPU frequency to optimize power via DVFS. While these efforts operate at a fine granularity, they focus on either SM allocation or frequency scaling on each GPU device. In contrast, OPSCALE is the first op-level LLM inference framework that addresses *system-level elasticity*, dynamically determining the optimal operator provisioning and scaling plan at a cluster scale.

8 Conclusion

OPSCALE rethinks serving granularity by shifting from monolithic replicas to operator-level autoscaling. Exploiting operator heterogeneity enables fine-grained resource provisioning, reducing scaling latency to sub-second scales. On production traces, OPSCALE maintains strict SLOs, while reducing GPU allocation by up to 36.3% and power usage by up to 28%.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024.
- [2] Amey Agrawal, Haoran Qiu, Junda Chen, Íñigo Goiri, Chaojie Zhang, Rayyan Shahid, Ramachandran Ramjee, Alexey Tumanov, and Esha Choukse. Medha: Efficiently serving multi-million context length LLM inference requests without approximations. *arXiv preprint arXiv:2409.17264*, 2024.
- [3] Benjamin Spector, Jordan Juravsky, Stuart Sul, Owen Dugan, Dylan Lim, Dan Fu, Simran Arora, Chris Ré. Look Ma, No Bubbles! Designing a Low-Latency Megakernel for Llama-1B. Accessed from <https://hazyresearch.stanford.edu/blog/2025-05-27-no-bubbles>, 2025.
- [4] Thomas Bonald and James W Roberts. Internet and the Erlang formula. *ACM SIGCOMM Computer Communication Review*, 42(1):23–30, 2012.
- [5] Cheng-Shang Chang. Stability, queue length, and delay of deterministic and stochastic queueing networks. *IEEE Transactions on Automatic Control*, 39(5):913–931, 1994.
- [6] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of The 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, 2018.
- [7] Yukang Chen, Weihao Cui, Han Zhao, Ziyi Xu, Xiaoze Fan, Xusheng Chen, Yangjie Zhou, Shixuan Sun, Bingsheng He, and Quan Chen. Towards high-goodput LLM serving with prefill-decode multiplexing. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2026)*, pages 2030–2047, 2026.
- [8] Xinhao Cheng, Zhihao Zhang, Yu Zhou, Jianan Ji, Jinchun Jiang, Zepeng Zhao, Ziruo Xiao, Zihao Ye, Yingyi Huang, Ruihang Lai, et al. Mirage Persistent Kernel: A Compiler and Runtime for Mega-Kernelizing Tensor Programs. *arXiv preprint arXiv:2512.22219*, 2025.
- [9] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. Serving heterogeneous machine learning models on multi-GPU servers with spatio-temporal sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 2022)*, pages 199–216, 2022.
- [10] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. PaLM: Scaling language modeling with Pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.
- [11] Patrick H Coppock, Brian Zhang, Eliot H Solomon, Vasilis Kypriotis, Leon Yang, Bikash Sharma, Dan Schatzberg, Todd C Mowry, and Dimitrios Skarlatos. LithOS: An operating system for efficient machine learning on GPUs. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP 2025)*, pages 1–17, 2025.
- [12] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems (NeurIPS 22)*, 35:16344–16359, 2022.
- [13] Aditya Dhakal, Sameer G Kulkarni, and KK Ramakrishnan. GSLICE: controlled spatial sharing of GPUs for a scalable inference platform. In *Proceedings of the 11th ACM Symposium on Cloud Computing (SoCC 2020)*, pages 492–506, 2020.
- [14] Xianzhe Dong, Tongxuan Liu, Yuting Zeng, Liangyu Liu, Yang Liu, Siyu Wu, Yu Wu, Hailong Yang, Ke Zhang, and Jing Li. HydraInfer: Hybrid disaggregated scheduling for multimodal large language model serving. *arXiv preprint arXiv:2505.12658*, 2025.

- [15] Jiangfei Duan, Runyu Lu, Haojie Duanmu, Xiuhong Li, Xingcheng Zhang, Dahua Lin, Ion Stoica, and Hao Zhang. MuxServe: Flexible spatial-temporal multiplexing for multiple LLM serving. *arXiv preprint arXiv:2404.02015*, 2024.
- [16] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. ServerlessLLM: low-latency serverless inference for large language models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 135–153, 2024.
- [17] Alim Ul Gias, Giuliano Casale, and Murray Woodside. ATOM: Model-driven autoscaling for microservices. In *Proceedings of the IEEE 39th International Conference on Distributed Computing Systems (ICDCS 2019)*, pages 1994–2004. IEEE, 2019.
- [18] GitHub. Nano-vLLM. <https://github.com/GeeekExplorer/nano-vllm>, 2025.
- [19] GitHub. NVIDIA Dynamo. <https://github.com/ai-dynamo/dynamo>, 2025.
- [20] GitHub. vLLM Production Stack. <https://github.com/vllm-project/production-stack>, 2025.
- [21] Ke Hong, Lufang Chen, Zhong Wang, Xiuhong Li, Qiuli Mao, Jianping Ma, Chao Xiong, Guanyu Wu, Buhe Han, Guohao Dai, et al. Semi-PD: Towards efficient LLM serving via phase-wise disaggregated computation and unified storage. *arXiv preprint arXiv:2504.19867*, 2025.
- [22] Junhao Hu, Jiang Xu, Zhixia Liu, Yulong He, Yuetao Chen, Hao Xu, Jiang Liu, Jie Meng, Baoquan Zhang, Shining Wan, et al. DEEPSERVE: Serverless large language model serving at scale. In *Proceedings of the 2025 USENIX Annual Technical Conference (ATC 2025)*, 2025.
- [23] Heyang Huang, Cunchen Hu, Jiaqi Zhu, Ziyuan Gao, Liangliang Xu, Yizhou Shan, Yungang Bao, Sun Ninghui, Tianwei Zhang, and Sa Wang. DDiT: Dynamic resource allocation for diffusion transformer model serving. *arXiv preprint arXiv:2506.13497*, 2025.
- [24] Wenhao Huang, Zhaolin Duan, Laiping Zhao, Yuhao Zhang, Yanjie Wang, Yiming Li, Yihan Wang, Yichi Chen, Zhihang Tang, Kang Chen, et al. μ Share: Non-intrusive kernel co-locating on NVIDIA GPUs. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA 2026)*, pages 1–14. IEEE, 2026.
- [25] Hugging Face. Llama-3-8b. <https://huggingface.co/meta-llama/Meta-Llama-3-8B>, 2025.
- [26] Hugging Face. Mixtral-8x7B-v0.1. <https://huggingface.co/mistralai/Mixtral-8x7B-v0.1>, 2025.
- [27] Hugging Face. Qwen2-57B-A14B. <https://huggingface.co/Qwen/Qwen2-57B-A14B-Instruct>, 2025.
- [28] Hugging Face. QWen2-7B-Instruct. <https://huggingface.co/Qwen/Qwen2-7B-Instruct>, 2025.
- [29] Hugging Face. Qwen2.5-VL-32B. <https://huggingface.co/Qwen/Qwen2.5-VL-32B-Instruct>, 2025.
- [30] Shashwat Jaiswal, Kunal Jain, Yogesh Simmhan, Anjaly Parayil, Ankur Mallick, Rujia Wang, Renee St. Amant, Chetan Bansal, Victor Ruhle, Anoop Kulkarni, Steve Kofsky, and Saravan Rajmohan. SAGESERVE: Optimizing LLM serving on cloud data centers with forecast aware auto-scaling. *Proceedings of ACM Measurements and Analysis of Computer Systems*, 9(3), December 2025.
- [31] Aditya K Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. Pod-Attention: Unlocking full prefill-decode overlap for faster LLM inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 25)*, pages 897–912, 2025.
- [32] Peter C Kiessler. A simulation analysis of sojourn times in a Jackson network. Technical report, 1980.
- [33] Kubernetes. Horizontal Pod Autoscaling. <http://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale>, 2026.
- [34] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 663–679, 2023.
- [35] Zejia Lin, Hongxin Xu, Guanyi Chen, Xianwei Zhang, and Yutong Lu. Bullet: Boosting GPU utilization for LLM serving via dynamic spatial-temporal orchestration. *arXiv preprint arXiv:2504.19516*, 2025.
- [36] Ziming Liu, Boyu Tian, Guoteng Wang, Zhen Jiang, Peng Sun, Zhenhua Han, Tian Tang, Xiaohe Hu,

- Yanmin Jia, Yan Zhang, et al. Expert-as-a-service: Towards efficient, scalable, and robust large-scale MoE serving. *arXiv preprint arXiv:2509.17863*, 2025.
- [37] Microsoft Azure. Azure VM NDm-A100-v4 sizes series. <https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/gpu-accelerated/ndma100v4-series>, 2024.
- [38] Microsoft Azure. Azure VM ND GB200-v6 sizes series. <https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/gpu-accelerated/nd-gb200-v6-series>, 2026.
- [39] NVIDIA. Documentation on NVIDIA Multi-Instance GPU (MIG). <https://www.nvidia.com/en-us/technologies/multi-instance-gpu/>, 2025.
- [40] NVIDIA. Documentation on NVIDIA Multi-Process Service (MPS). <https://docs.nvidia.com/deploy/mps/index.html>, 2025.
- [41] NVIDIA. Nsight Systems. <https://developer.nvidia.com/nsight-systems>, 2025.
- [42] NVIDIA. NVIDIA DCGM. <https://developer.nvidia.com/dcgm>, 2025.
- [43] NVIDIA. NVIDIA Green Context Documentation. https://docs.nvidia.com/cuda/cuda-driver-api/group_CUDA_GREEN_CONTEXTS.html, 2025.
- [44] OpenAI. Introducing ChatGPT. <https://openai.com/index/chatgpt/>, 2022.
- [45] OpenAI. ChatGPT Codex. <https://chatgpt.com/features/codex>, 2025.
- [46] OpenAI. Introducing Deep Research. <https://openai.com/index/introducing-deep-research/>, 2025.
- [47] Melissa Z Pan, Negar Arabzadeh, Riccardo Cogo, Yuxuan Zhu, Alexander Xiong, Lakshya A Agrawal, Huanzhi Mao, Emma Shen, Sid Pallerla, Liana Patel, et al. Measuring agents in production. *arXiv preprint arXiv:2512.04123*, 2025.
- [48] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA 24)*, pages 118–132. IEEE, 2024.
- [49] Archit Patke, Dharmath Reddy, Saurabh Jha, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. Hierarchical autoscaling for large language model serving with Chiron. *arXiv preprint arXiv:2501.08090*, 2025.
- [50] Yifan Qiao, Shu Anzai, Shan Yu, Haoran Ma, Shuo Yang, Yang Wang, Miryung Kim, Yongji Wu, Yang Zhou, Jiarong Xing, Joseph E. Gonzalez, Ion Stoica, and Harry Xu. ConServe: Fine-grained GPU harvesting for LLM online and offline co-serving, 2025.
- [51] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. *arXiv preprint arXiv:2407.00079*, 2024.
- [52] Haoran Qiu, Subho S Banerjee, Saurabh Jha, Zbigniew T Kalbarczyk, and Ravishankar K Iyer. FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices. In *Proceedings of the 14th USENIX symposium on operating systems design and implementation (OSDI 2020)*, pages 805–825, 2020.
- [53] Haoran Qiu, Anish Biswas, Zihan Zhao, Jayashree Mohan, Alind Khare, Esha Choukse, Íñigo Goiri, Zeyu Zhang, Haiying Shen, Chetan Bansal, et al. ModServe: Scalable and resource-efficient large multimodal model serving. *arXiv preprint arXiv:2502.00937*, 2025.
- [54] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. Power-aware deep learning model serving with μ -Serve. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 75–93, 2024.
- [55] Sudipta Saha Shubha, Haiying Shen, and Anand Iyer. USHER: Holistic interference avoidance for resource optimized ML inference. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 947–964, 2024.
- [56] Gursimran Singh, Xinglu Wang, Ivan Hu, Timothy Yu, Linzi Xing, Wei Jiang, Zhefeng Wang, Xiaolong Bai, Yi Li, Ying Xiong, et al. Efficiently serving large multimedia models using EPD disaggregation. *arXiv preprint arXiv:2501.05460*, 2024.
- [57] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *Proceedings of the 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA 25)*, pages 1348–1362. IEEE, 2025.

- [58] Foteini Strati, Xianzhe Ma, and Ana Klimovic. Orion: Interference-aware, fine-grained GPU sharing for ML applications. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys 2024)*, pages 1075–1092, 2024.
- [59] The AIBrix Team, Jiaxin Shan, Varun Gupta, Le Xu, Haiyang Shi, Jingyuan Zhang, Ning Wang, Linhui Xu, Rong Kang, Tongping Liu, et al. AIBrix: Towards scalable, cost-effective large language model inference infrastructure. *arXiv preprint arXiv:2504.03648*, 2025.
- [60] vLLM. Distributed Inference and Serving. https://docs.vllm.ai/en/latest/serving/distributed_serving.html, 2024.
- [61] vLLM. vLLM Profiler. <https://docs.vllm.ai/en/stable/contributing/profiling/>, 2025.
- [62] Bin Wang, Bojun Wang, Changyi Wan, Guanzhe Huang, Hanpeng Hu, Haonan Jia, Hao Nie, Mingliang Li, Nuo Chen, Siyu Chen, et al. Step-3 is large yet affordable: Model-system co-design for cost-effective decoding. *arXiv preprint arXiv:2507.19427*, 2025.
- [63] Zibo Wang, Pinghe Li, Chieh-Jan Mike Liang, Feng Wu, and Francis Y Yan. Autothrottle: A practical bi-level approach to resource management for SLO-targeted microservices. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 2024)*, pages 149–165, 2024.
- [64] Ziliang Wang, Shiyi Zhu, Jianguo Li, Wei Jiang, Kadangode K Ramakrishnan, Yangfei Zheng, Meng Yan, Xiaohong Zhang, and Alex X Liu. DeepScaling: microservices autoscaling for stable cpu utilization in large scale cloud systems. In *Proceedings of the 13th Symposium on Cloud Computing (SoCC 2022)*, pages 16–30, 2022.
- [65] Yuxing Xiang, Xue Li, Kun Qian, Yufan Yang, Diwen Zhu, Wenyan Yu, Ennan Zhai, Xuanzhe Liu, Xin Jin, and Jingren Zhou. Aegaeon: Effective GPU pooling for concurrent LLM serving on the market. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 1030–1045, 2025.
- [66] Jiarong Xing, Yifan Qiao, Simon Mo, Xingqi Cui, Gur-Eyal Sela, Yang Zhou, Joseph Gonzalez, and Ion Stoica. Towards efficient and practical GPU multitasking in the era of LLM. *arXiv preprint arXiv:2508.08448*, 2025.
- [67] Shan Yu, Jiarong Xing, Yifan Qiao, Mingyuan Ma, Yangmin Li, Yang Wang, Shuo Yang, Zhiqiang Xie, Shiyi Cao, Ke Bao, et al. Prism: Unleashing GPU sharing for cost-efficient multi-LLM serving. *arXiv preprint arXiv:2505.04021*, 2025.
- [68] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.
- [69] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, et al. NanoFlow: Towards optimal large language model serving throughput. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 749–765, 2025.
- [70] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, et al. MegaScale-Infer: Serving mixture-of-experts at scale with disaggregated expert parallelism. *arXiv preprint arXiv:2504.02263*, 2025.
- [71] Pengfei Zuo, Huimin Lin, Junbo Deng, Nan Zou, Xingkun Yang, Yingyu Diao, Weifeng Gao, Ke Xu, Zhangyu Chen, Shirui Lu, et al. Serving large language models on Huawei CloudMatrix384. *arXiv preprint arXiv:2506.12708*, 2025.

Algorithm 1 Greedy Operator Provisioning Algorithm

Require: DAG $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, QPS q , batch limits $B_v^{\max}$, parallelism sets $\mathcal{P}_v$, latency functions $T_v(b, p)$ from profiling, SLO T_{slo} with a buffer ϵ

- 1: Initialize $p_v \leftarrow \min \mathcal{P}_v$ and $b_v \leftarrow 1$ for all $v \in \mathcal{V}$
- 2: **for all** $v \in \mathcal{V}$ **do** ▷ Per-operator initialization
- 3: $\lambda \leftarrow \text{ARRIVALRATES}(\mathcal{G}, q, b_v)$
- 4: $r_v \leftarrow \lceil \lambda / \mu_v(1, p_v) \rceil$ where $\mu_v(b, p) = b / T_v(b, p)$
- 5: $(b_v, r_v) \leftarrow \arg \min_{b \in \{1, \dots, B_v^{\max}\}} s_v(\lambda, \lceil \lambda / \mu_v(b, p_v) \rceil, b, p_v)$
- 6: **end for**
- 7: Recompute $\lambda \leftarrow \text{ARRIVALRATES}(\mathcal{G}, q, \{b_v\})$
- 8: $s_v \leftarrow W_v(\lambda_v, r_v, \mu_v(b_v, p_v)) + T_v(b_v, p_v) / b_v$ for all v
- 9: $T \leftarrow \text{CRITICALPATHLATENCY}(\mathcal{G}, \{s_v\})$
- 10: **while true do**
- 11: **if** $T \leq T_{slo} - \epsilon$ **then** ▷ Scaling down
- 12: $j \leftarrow \text{BOTTLENECKONCRITICALPATH}(\mathcal{G}, \{s_v\})$
- 13: $\mathcal{M} \leftarrow \{(r_j - 1, b_j, p_j)\} \cup \{(r_j - 1, b, p_j) \mid b \in [b_j, B_j^{\max}]\} \cup \{(r_j - 1, b, p) \mid b \in [b_j, B_j^{\max}], p \in \mathcal{P}_j\}$
- 14: Filter $\mathcal{M}$ with stability check $\lambda_j < (r'_j) \mu_j(b', p')$
- 15: For each $m \in \mathcal{M}$: tentatively recompute T'
- 16: Choose $m^* \in \arg \min \{\text{COST}(\mathbf{r}', \mathbf{p}') \mid T' \leq T_{slo}\}$
- 17: **if** m^* exists **then** apply m^* , set $T \leftarrow T'$, continue
- 18: **else break** ▷ No further safe downscale
- 19: **end if**
- 20: **else if** $T > T_{slo}$ **then** ▷ Scaling up
- 21: $j \leftarrow \text{BOTTLENECKONCRITICALPATH}(\mathcal{G}, \{s_v\})$
- 22: $\mathcal{M} \leftarrow \{(r_j + 1, b_j, p_j)\} \cup \{(r_j + 1, b, p_j) \mid b \in [1, B_j^{\max}]\} \cup \{(r_j + 1, b, p) \mid b \in [1, B_j^{\max}], p \in \mathcal{P}_j\}$
- 23: For each $m \in \mathcal{M}$: tentatively re-evaluate T'
- 24: Choose $m^* \in \arg \max \{T - T'\}$; Prefer the smallest Δr_j that achieves $T' \leq T_{slo}$
- 25: **if** m^* exists **then** apply m^* , set $T \leftarrow T'$, continue
- 26: **else break** ▷ Cannot improve further
- 27: **end if**
- 28: **else break** ▷ Within tolerance of SLO
- 29: **end if**
- 30: **end while**
- 31: **return** $\{(r_v, b_v, p_v)\}_{v \in \mathcal{V}}$

A Complementary Analysis Figures

Figure 20 and Figure 21 show details of operator compute and memory sensitivities to sequence length across different model architectures (complementary to Figure 4). Figure 22 separately plots GPU savings, energy savings, and memory savings for op-level autoscaling under varying sequence lengths. Figure 23 separately plots GPU savings, energy savings, and memory savings for op-level autoscaling under varying QPS (complementary to Figure 8 and 9).

B Algorithm Pseudocode

Algorithm 1 presents the pseudocode for operator provisioning algorithm (§4.3). Algorithm 2 presents the pseudocode

Algorithm 2 Greedy Op-Level Placement

Require: DAG $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, config $\{(r_v, b_v, p_v)\}_{v \in \mathcal{V}}$ from Alg. 1, device set $\mathcal{D}$, device capacities $\{M_d, U_d\}_{d \in \mathcal{D}}$, interference model $I_{d,v}(b, p) \geq 1$ (from profiling).

- 1: $k_{\text{base}} \leftarrow \min_{v \in \mathcal{V}} r_v$ ▷ Number of full model instances
- 2: Construct replica sets:
- 3: $\mathcal{R}_{\text{base}} \leftarrow \{(v, i) \mid v \in \mathcal{V}, i \in [1, k_{\text{base}}]\}$
- 4: $\mathcal{R}_{\text{extra}} \leftarrow \{(v, i) \mid v \in \mathcal{V}, i \in [k_{\text{base}} + 1, r_v]\}$
- 5: Sort $(v, k) \in \mathcal{R}_{\text{extra}}$ in descending order of T_v
- 6: $\mathcal{D}_{\text{base}} \leftarrow \text{DEPLOYMODELINSTANCE}(\mathcal{R}_{\text{base}})$
- 7: $\mathcal{D}_{\text{empty}} \leftarrow \mathcal{D} \setminus \mathcal{D}_{\text{base}}$
- 8: **for all** $(v, k) \in \mathcal{R}_{\text{extra}}$ **do**
- 9: $\text{Candidates} \leftarrow \emptyset$
- 10: **for all** $d \in \mathcal{D}_{\text{base}}$ **do** ▷ Try existing devices first
- 11: **if** $\text{MemLoad}_d + M_v > M_d$ **then**
- 12: **continue**
- 13: **end if**
- 14: $T'_v \leftarrow T_v \cdot I_{d,v}(b_v, p_v)$
- 15: **if** $\text{RECOMPUTELATENCY}(\mathcal{G}) > T_{slo}$ **then**
- 16: **continue**
- 17: **end if**
- 18: $\text{slack}_{\text{mem}} \leftarrow M_d - (\text{MemLoad}_d + M_v)$
- 19: $\text{slack}_{\text{comp}} \leftarrow U_d - (\text{CompLoad}_d + T'_v)$
- 20: $\text{Candidates}.\text{APPEND}(d)$
- 21: **end for**
- 22: **if** $\text{Candidates} == \emptyset$ **then** ▷ No existing device fits
- 23: $d_{\text{new}} \leftarrow \text{PROVISIONDEVICE}(\mathcal{D}_{\text{empty}}, \mathcal{D}_{\text{base}})$
- 24: $d^* \leftarrow d_{\text{new}}$
- 25: **else**
- 26: $\text{slack} \leftarrow \text{COMPUTEWEIGHTEDSLACK}(\text{Candidates})$
- 27: $d^* \leftarrow \arg \max_{d \in \text{Candidates}} \text{slack}$
- 28: **end if**
- 29: $\text{Placement} \leftarrow \text{ASSIGNOPERATOR}(v, k, d^*)$
- 30: **end for**
- 31: **return** Placement

for operator-to-device placement algorithm (§4.4).

C Additional Operator Characterizations

§2.1 presents the characterization study on operator heterogeneity under inference workload changes, and it focuses on compute time, memory usage, and SM core allocation sensitivities. This section analyzes additional types of sensitivities.

Queueing Characteristics. Building on the per-operator compute sensitivities, we analyze how operators respond to increasing request load (RPS) using M/M/c queueing theory. Each operator is modeled as a multi-replica queueing system, where the service rate is $\mu = 1/(\text{op_latency} \cdot \text{num_layers})$, derived from the measured GPU execution times, and the arrival rate is $\lambda = \text{requests_per_second} / \text{batch_size}$. Using the Erlang-C formula, we estimate waiting times and determine the minimum number of replicas required to maintain system stability under varying RPS.

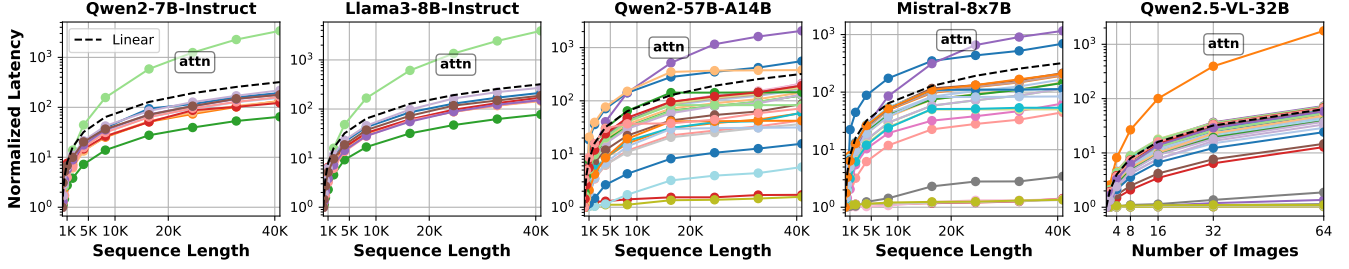


Figure 20: Compute sensitivities to input data size, for various operators in different model architectures.

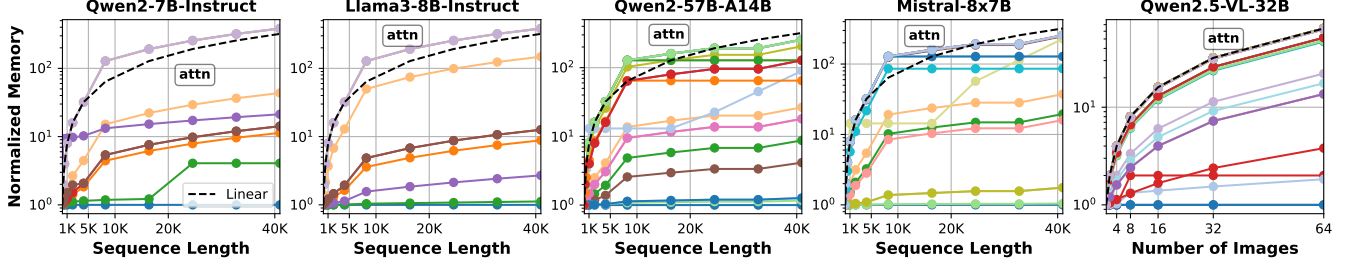


Figure 21: Memory sensitivities to input data size, for various operators in different model architectures.

Figure 24 shows heterogeneous queueing sensitivities across operators that closely reflect their compute characteristics. For attention operators, especially at longer sequence lengths, the number of replicas required grows sharply with increasing RPS, reflecting their high per-token computational cost. In MoE models like Mixtral, the FusedMoE linear operator dominates compute at short sequence lengths, leading to a pronounced scaling of replicas even for relatively small sequences. In contrast, lighter operators, such as layer norms and embeddings, exhibit moderate sensitivity to RPS. Queueing delays increase non-linearly when replication is insufficient, so small reductions in replicas can cause disproportionately high waiting times. These observations highlight that combining compute profiling and queueing modeling enables precise, operator-specific replication strategies by selectively replicating high-demanding operators (*e.g.*, attention) while avoiding overprovisioning lightweight ones, thereby meeting end-to-end latency and throughput targets efficiently.

Takeaway 8: Operators exhibit diverse queueing sensitivity with increasing load. Insufficient replication causes non-linear queueing delays, emphasizing the need for operator-specific replication strategies.

Dataflow Characteristics. We analyze data flows by quantifying the communication payload between adjacent operators in the model graph. Specifically, we perform transient memory profiling to capture how each operator’s input-output scale with sequence length and batch size, revealing that most operators exhibit linear growth in data volume (as shown in Figure 25). Communication overhead is estimated from trans-

Table 3: Models in characterization study and evaluations.

Model	Model Size	Modality	Architecture
Qwen2-7B [28]	7B	Text	Dense LLM
Qwen2-MoE [27]	57B (14B)	Text	MoE LLM
Llama3-8B [25]	8B	Text	Dense LLM
Mixtral-8×7B [26]	47B (13B)	Text	MoE LLM
Qwen2.5-VL-32B [29]	32B	Visual	Encoder+LLM

fer latency, which scales proportionally with payload size. While many operators (*e.g.*, element-wise functions) have near-constant per-request data volume, others (*e.g.*, attention and linear operators) scale with sequence length. Comparative analysis of compute versus NVLink transfer time shows transfer overhead reaching $\sim 20\%$ for certain operators (*e.g.*, SiLu Mul) but remaining below 5% for most operators.

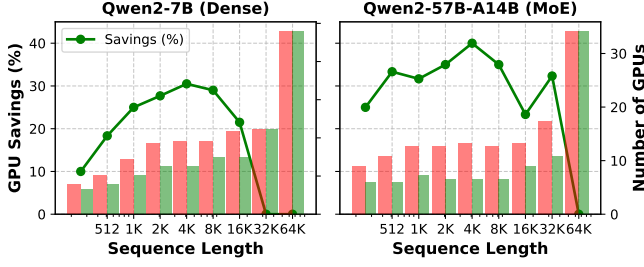
Takeaway 9: Data volume scales linearly or remains flat with sequence length across operators. Transfer overhead can reach 20% of compute time, making transfer costs non-trivial when placing operators across devices.

D Model Details

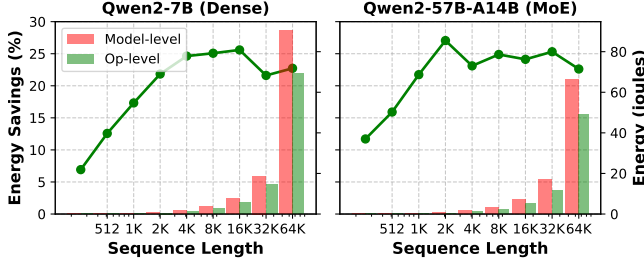
Table 3 details the models used in our characterization study and evaluations. We consider models of two dominant architectures: dense LLMs and mixture-of-experts (MoE) models.

E Prefill vs. Decode

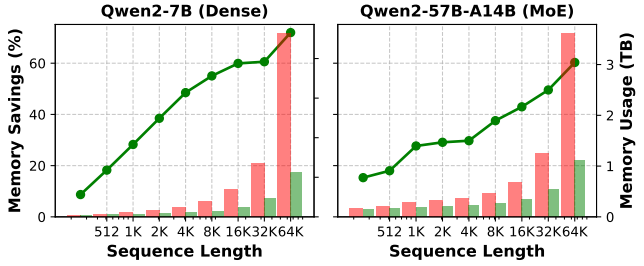
We compare op-level and model-level provisioning behavior across the prefill (known to be compute-bound [48]) and decode stages (memory-bound), illustrating how their distinct



(a) GPU savings.



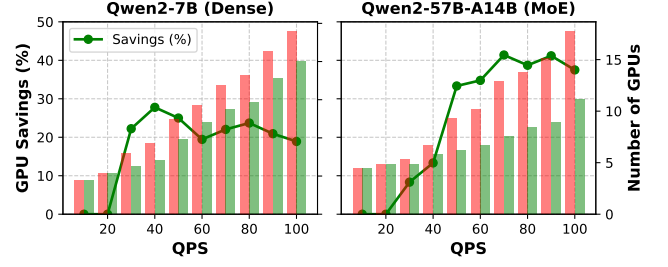
(b) Energy savings.



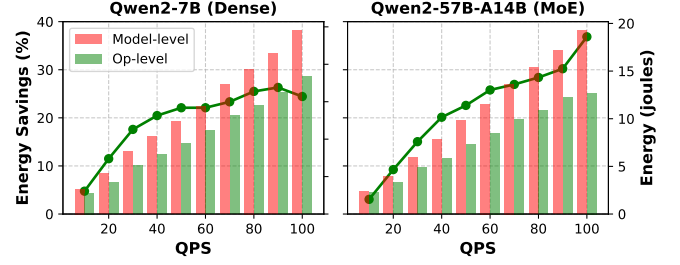
(c) Memory savings.

Figure 22: Benefits of operator-level resource management under varying sequence lengths. Left Y-axis denotes saving percentages. Right Y-axis denotes the absolute GPU usage, energy consumption, and memory footprint.

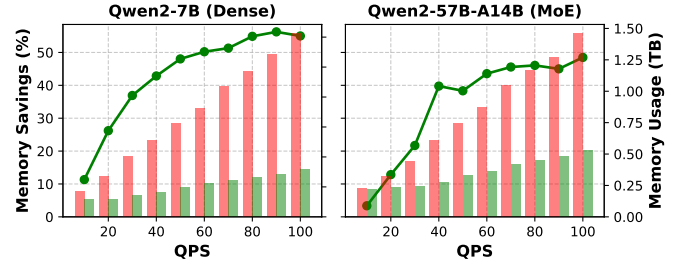
computational and temporal characteristics drive different scaling needs. We analyze two production-scale LLM inference traces from (1) Azure LLM inference cluster [57] and (2) Mooncake LLM serving platform [51]. As shown in Figure 26, op-level resource management yields consistently higher savings during the prefill stage across all workloads. On Azure traces, prefill achieves up to 35% GPU, 25% energy, and 45% memory savings for chat services, while decode savings remain modest (below 15%). Savings in coding services are lower due to their low QPS in the traces. Similarly, in Mooncake traces, prefill achieves up to 22% GPU savings, 20% energy, and 41% memory, higher than the savings achieved during decode. These results highlight that prefill stages benefit more from op-level provisioning and scaling due to their denser compute utilization and shorter execution bursts.



(a) GPU savings.



(b) Energy savings.



(c) Memory savings.

Figure 23: Benefits of operator-level resource management under varying QPS. Left Y-axis denotes saving percentages. Right Y-axis denotes the absolute GPU usage, energy consumption, and memory footprint.

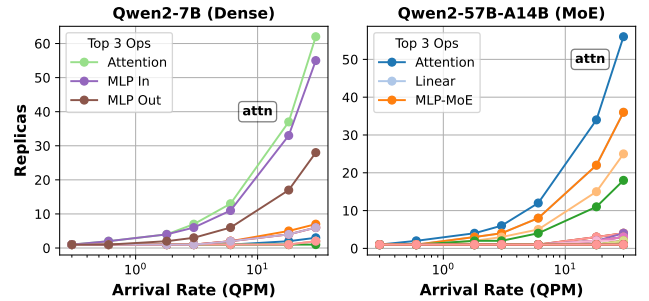


Figure 24: Queueing sensitivity to arrival rate.

Insight 1: Prefill stages offer substantially higher optimization potential than decode—up to 2–3× greater resource savings, as they are more compute-intensive and bursty, making them ideal targets for fine-grained op-level model provisioning and scaling.

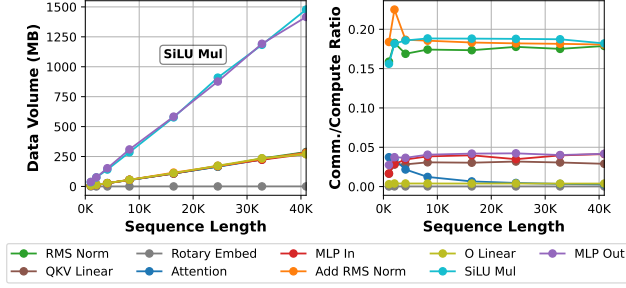


Figure 25: Operator input data volume for Qwen2-7B.

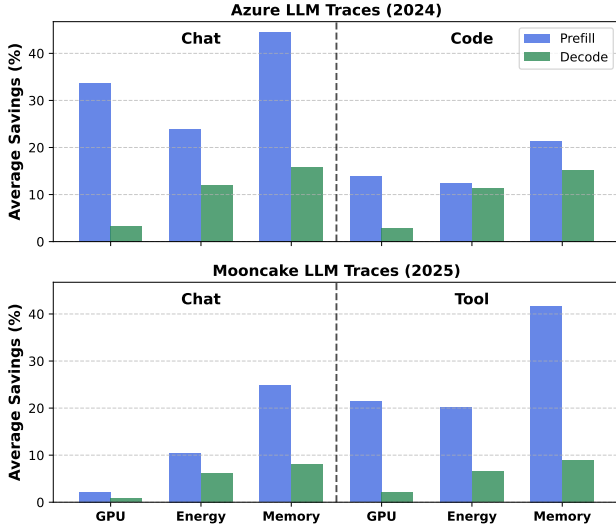


Figure 26: Benefits of operator-level resource management in prefill vs. decode stages for Qwen2-7B.

F Additional Evaluation Results

This section provides detailed evaluation results for the Mixture-of-Experts (MoE) model, Qwen2-57B-A14B. Due to the high operator heterogeneity in MoE architectures (§2.1), monolithic scaling often results in significant resource over-provisioning. Our results demonstrate that OPSCALE’s op-level elasticity is particularly effective for such models, with higher gains compared to serving dense LLMs.

MoE Autoscaling Performance. Figure 27 illustrates the real-time autoscaling behavior over a 1-hour trace. While all systems aim to keep the P99 TTFT below the 2.0s SLO, the resource footprints vary significantly. OPSCALE maintains a much tighter envelope around the actual workload demand. As shown in the middle row of Figure 27, OPSCALE (green line) operates with a significantly lower number of GPUs throughout the trace, peaking at roughly 25 GPUs during high-load periods, whereas AIBrix and DynamoLLM frequently exceed 30–35 GPUs to meet the same latency targets.

Crucially, even with fewer resources, OPSCALE maintains a more stable P99 TTFT. While the Production Stack and

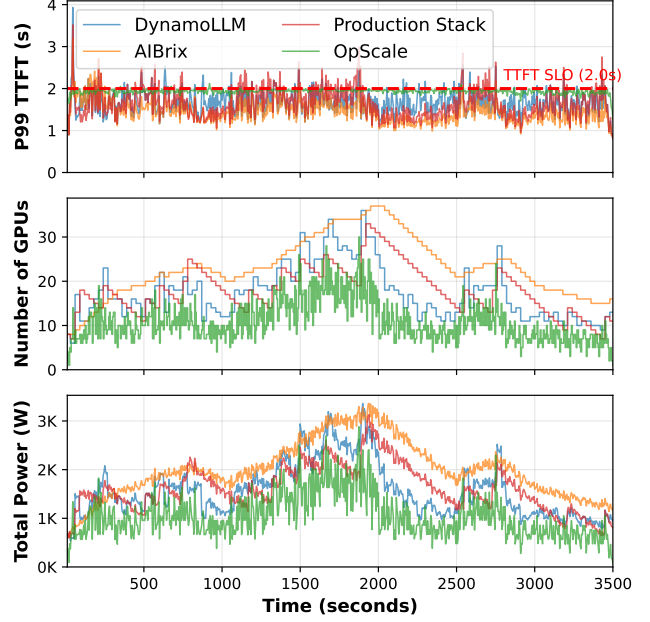


Figure 27: Comparison of GPU usage, request performance (P99 TTFT), and total power usage during autoscaling for Qwen2-57B-A14B (MoE).

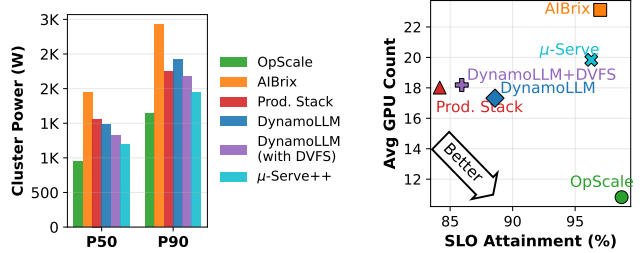


Figure 28: Cluster-level power consumption comparison.

Figure 29: Cost vs. SLO attainment trade-off.

AIBrix show frequent spikes that touch or exceed the 2.0s SLO line, OPSCALE remains consistently below the threshold. This efficiency is attributed to the system’s ability to scale only the bottlenecked operators rather than replicating the entire 57B parameter model.

Power and Cost Trade-offs for MoE. The power consumption benefits observed in the dense model are further amplified in the MoE setting. Figure 28 shows the cluster-level power consumption at P50 and P90 levels. OPSCALE with DVFS achieves the lowest power profile, outperforming μ -Serve++ by approximately 15% at the P90 mark. Notably, even without DVFS, the base OPSCALE implementation consumes less power than the Production Stack and AIBrix, confirming that reducing the active GPU count through fine-grained scaling is more impactful than model-level power management.

Finally, Figure 29 summarizes the cost-performance Pareto

frontier. OPSCALE occupies the bottom-right corner of the plot, representing the ideal high-SLO-attainment, low-GPU-count quadrant. Compared to AIBrix, OPSCALE reduces the average GPU count from ~ 23 to ~ 11 while simultaneously improving SLO attainment from $\sim 97\%$ to over 98% . While μ -Serve++ and DynamoLLM+DVFS improve upon their respective vanilla baselines, they still require 18–20 GPUs on average. This highlights that for MoE models, the overhead of monolithic replicas is a primary bottleneck that cannot be fully mitigated by frequency scaling or partitioning alone. These results confirm that operator-level elasticity allows OPSCALE to bypass the all-or-nothing scaling penalty inherent in MoE models, delivering superior performance with a substantially smaller hardware footprint.